Riassunto basi di dati

Dimix

2025-03-13

Indice

Lezione 1 – Introduzione **4**
1 Introduzione alle basi di dati . . . 4
1.1 Definizione e contesto . . . 4
1.2 Considerazioni generali sull'uso delle basi di dati . . . 4
2 I sistemi di gestione delle basi di dati . . . 5
2.1 Modelli dei dati . . . 5
2.2 Architettura di un DBMS . . . 5
2.3 Le categorie di linguaggi nei DBMS . . . 5
2.4 Utenze di un DBMS . . . 5

Lezione 2 – Il modello relazionale **7**
1 Il modello relazionale . . . 7
1.1 Definizione . . . 7
1.2 Proprietà delle relazioni . . . 8
2 L'informazione incompleta . . . 8
2.1 Cause e soluzioni . . . 8
3 Vincoli di integrità . . . 8
3.1 Vincoli di integrità . . . 8
3.2 I vincoli di chiave . . . 9
3.3 L'integrità referenziale . . . 9
4 Forme di relazione . . . 10
4.1 Prima forma normale 1NF . . . 11
4.2 Seconda forma normale 2NF . . . 11
4.3 Terza forma normale 3NF . . . 12
4.4 Forma normale di Boyce-Codd BCNF . . . 13
5 Proprietà delle decomposizioni . . . 13
5.1 Proprietà . . . 13
5.2 Algoritmi di progettazione . . . 14

Lezione 3 - Modellazione concettuale **15**
1 Sistemi di basi di dati . . . 15
1.1 Criteri di progettazione . . . 15
2 Progettazione concettuale . . . 16
2.1 Il modello ER . . . 16
2.2 Entità, relazioni e attributi . . . 16
2.3 Cardinalità . . . 17
2.4 Identificatori . . . 18
2.5 Gerarchie di generalizzazione . . . 19

Lezione 4 – Progettazione Logica e SQL DDL **21**

1 Progettazione logica . . . 21
1.1 Ristrutturazione dello schema ER . . . 21
1.2 Traduzione dello schema ER . . . 24
1.3 Reverse engineering . . . 26
2 Creazione dello schema relazione con SQL DDL . . . 26
2.1 Introduzione a SQL . . . 26
2.2 Domini . . . 26
2.3 Creazione dello schema . . . 26
2.4 Vincoli . . . 27
2.5 Modifica dello schema . . . 28

Lezione 5 – L'algebra relazionale TO COMPLETE/REVIEW **30**
Operatori dell'algebra relazione . . . 30
Operatori insiemistici . . . 30
Operatori unari . . . 30
Operatori binari . . . 30
Operatori Join . . . 30
Divisione . . . 31
Equivalenza di espressioni algebriche . . . 31

Lezione 6 – SQL **33**
Introduzione . . . 33
Interrogazioni . . . 33
Join . . . 35
Interrogazioni complesse . . . 36
Interrogazioni di tipo insiemistico . . . 37
Interrogazioni nidificate . . . 37
Piano di esecuzione delle interrogazioni . . . 39
Interrogazioni avanzate TO COMPLETE . . . 39
Viste TO COMPLETE . . . 40
Trigger e basi di dati attive TO DO . . . 41

Lezione 7 - Progettazione fisica **42**
1 Progettazione fisica . . . 42
1.1 Introduzione . . . 42
1.2 Gestione dei blocchi . . . 42
2 Indici . . . 44
2.1 Caratteristiche generali . . . 44
2.2 File ordinati . . . 44
2.3 Indici secondari . . . 46
3 Indici multilivello . . . 48
3.1 Introduzione . . . 48
3.2 Alberi B-tree . . . 50
3.3 Alberi B^+ . . . 51

Lezione 8 - Elementi di sicurezza **53**
1 Politiche e modelli . . . 53
1.1 Introduzione . . . 53
1.2 Politiche di sicurezza . . . 53
2 Modelli per il controllo dell'accesso . . . 56
2.1 System R . . . 56
2.2 Autorizzazioni su viste . . . 57

Lezione 9 - Cenni su transazioni **59**
1 Transazione . . . 59

1.1 Introduzione 59
1.2 Problemi di concorrenza 59

Lezione 1 – Introduzione

1 Introduzione alle basi di dati

1.1 Definizione e contesto

Una **base di dati** è una collezione di dati correlati. Per **dato** intendiamo la registrazione simbolica di un elemento della realtà.

Una base di dati possiede alcune caratteristiche distintive:

- Rappresenta un aspetto limitato della realtà, riferito a un sottoinsieme dei dati disponibili denominato **universo del discorso**.
- Costituisce un insieme di dati logicamente coerenti e dotati di un significato intrinseco.
- È progettata, costruita e popolata per uno scopo specifico, rivolto a un gruppo di utenti determinato e ad alcune specifiche applicazioni.

In una base di dati i dati sono correlati per fornire un'interpretazione e trasformarli in **informazione**.

Esempi:

- Dati non correlati: `Roma, 41.89, 12.48`
- Dati correlati: `Roma → Posizione: Latitudine 41.89, Longitudine 12.48`

Esistono tre tipologie di dati

1. **Dati non strutturati**: tipicamente disponibili in formato testuale (memorizzati su file o sistemi di gestione documentale)
2. **Dati semi-strutturati**: tipicamente disponibili in formati come XML, HTML, JSON, RDF, presentano una informazione struttura ma con pochi vincoli di formato e di coerenza dei dati, insieme ad una frequente ridondanza.
3. **Dati strutturati**: tipicamente in formato tabellare, memorizzati in DBMS relazionali o fogli di calcolo come Excel, la loro struttura è rigida, definita da un insieme di campi che si applicano a tutti gli oggetti descritti ma non è detto che siano rispettati vincoli di formato, coerenza dei dati e che non vi sia ridondanza.

1.2 Considerazioni generali sull'uso delle basi di dati

Le basi di dati in confronto alla gestione applicativa

Ogni applicazione possiede un sistema di gestione dei dati, tipicamente in memoria e su file andando a definire di volta in volta le strutture dati necessarie e l'organizzazione dei file, utilizzare invece una base di dati per la gestione dei dati in ausilio delle applicazioni è una scelta che implica alcune caratteristiche specifiche nel modo in cui i dati vengono trattati rispetto alle applicazioni.

Le caratteristiche delle basi di dati

- **Autodescrizione**: Una base di dati contiene sia i dati che una descrizione completa della sua struttura e dei suoi vincoli, attraverso un catalogo.
- **Indipendenza tra programmi e dati**: La struttura di memorizzazione dei dati è astratta e coerente al modello dei dati usato dalla base di dati, in tutti i casi essa è indipendente dalle applicazioni
- **Viste multiple dei dati**: Una base di dati può fornire un accesso a sottoinsiemi e a forme di organizzazione diversi dei dati a diversi utenti e applicazioni per mezzo di viste virtuali
- **Condivisione e controllo della concorrenza**: una base di dati garantisce che utenti e applicazioni diverse possano gestire in modo concorrente gli stessi dati, garantendo al tempo stesso la loro coerenza e correttezza per metto di transazioni.

2 I sistemi di gestione delle basi di dati

2.1 Modelli dei dati

L'organizzare i dati in una base di dati richiede la definizione di un modello logico che detti le regole di organizzazione dei dati stessi.

Il modello dei dati è un insieme di costrutti, formalismi di rappresentazione e operazioni di base necessari alla realizzazione della descrizione astratta e indipendente dalla memorizzazione fisica dei dati, ai vincoli che ne garantiscono la coerenza e correttezza e alla loro gestione.

Il modello relazione è il modello dei dati più diffuso, esistono anche altri modelli come possono essere quelli *reticolari*, *gerarchici* e ad *oggetti*.

Le nozioni di schema e istanza

- **Schema** (Intensione): è un modello logico che definisce la struttura e i vincoli dei dati da rispettare (fisso nel tempo).
- **Istanza** (Estensione): insieme dei dati organizzati rispettando lo schema (varia nel tempo).

L'istanza della base di dati in un particolare istante di tempo è detta **stato della base di dati**.

2.2 Architettura di un DBMS

Un **DBMS (Database Management System)** è un insieme di programmi che consente la creazione e manutenzione di basi di dati, in particolare consente di definire, costruire, manipolare e condividere basi di dati per vari utenti e applicazioni.

Obiettivo del DBMS è gestire i dati in modo efficiente evitando duplicazioni, errori, e garantendo integrità e sicurezza.

Architettura a tre livelli

L'architettura concettuale di una base di dati è caratterizzata da tre livelli:
1. **Livello interno** (fisico): memorizzazione fisica dei dati.
2. **Livello logico**: rappresentazione logica della base di dati.
3. **Livello esterno**: viste personalizzate per gli utenti.

2.3 Le categorie di linguaggi nei DBMS

- **DDL (Data Definition Language)**: istruzioni per la creazione e manutenzione dello schema.
- **DML (Data Manipulation Language)**: istruzioni per la manutenzione e interrogazione dei dati.
- **DQL (Data Query Language)**: istruzioni per l'interrogazione dei dati di una base di dati.
- **DCL (Data Control Language)**: istruzioni per gestire permessi e i controlli di accesso alla base di dati.

2.4 Utenze di un DBMS

in una base di dati esistono due macrocategorie di utenti:

- **DBA User**: Utenti amministratori dotati di privilegi massimi rispetto ad una o più basi di dati, possono essere in possesso delle funzionalità DDL, DML, DQL, DCL, gestire i privilegi e la sicurezza, gestiscono l'ottimizzazione delle basi di dati e gestiscono lo spazio di memorizzazione fisica.
- **DB User**: utenti con privilegi specifici e limitati su una o più basi di dati, essi sono suddivisi in:
  - Developer user
  - Occasional user
  - Casual user

I DBMS adottano un'architettura di comunicazione client/server, l'accesso può avvenire da postazioni locali tramite pipe del sistema operativo o remote tramite socket TCP/IP.

Gli utenti del DMBS utilizzano un software client per comunicare con il server, questo software può essere basato su:

- Riga di comando
- Interfaccia grafica
- Interfaccia web

Lezione 2 – Il modello relazionale

1 Il modello relazionale

1.1 Definizione

Il **modello relazionale** fu proposto da Edgar Codd nel 1970 e introdotto nei DBMS nel 1981.

- Basato sulla nozione di **relazione matematica**, intesa come sottoinsieme del prodotto cartesiano tra insiemi di dati, detti domini.
- Definisce un insieme di vincoli sui dati.
- Associato al linguaggio dell'**algebra relazionale** per eseguire interrogazioni.

Tabella e domini

La rappresentazione più intuitiva di una relazione è la tabella, una base di dati relazionale è rappresentata da una collezione di tabelle.

Ogni tabella ha un nome unico nella base di dati, con le seguenti proprietà:

- Ogni **riga** rappresenta un record.
- Ogni **colonna** ha un nome distinto **attributo** A_k, associato a un insieme di valori D_k appartenenti ad un certo **dominio**

Per dominio viene intesa intesa una collezione di valori atomici, in termini pratici, i domini a partire dai quali sono costruite le relazioni nei DBMS sono definiti a partire da tipi di dati, come ad esempio stringhe, interi e date.

La tabella

Dati n domini $D_1, D_2, \cdots, D_n$, ogni riga di una tabella è una ennupla ordinata di valori $(d_1, d_2, \cdots, d_n)$ con d_k appartenente al dominio D_k del corrispondente attributo A_k.
Una tabella contiene un sottoinsieme di tutte le righe possibili, cioè un sottoinsieme del prodotto cartesiano $D_1 \times D_2 \times \cdots \times D_n$.

La relazione matematica

Siano $D_1, D_2, \cdots, D_n$ n insieme di valori anche non distinti, il prodotto cartesiano $D_1 \times D_2 \times \cdots \times D_n$ è definito cosi:

$$D_1 \times D_2 \times \cdots \times D_n = \{(d_1, d_2, \cdots, d_n) | d_1 \in D_1, d_2 \in D_2, \cdots, d_n \in D_n\}$$

Una relazione matematica R su $D_1, D_2, \cdots, D_n$ è cosi definita:

$$R \subseteq D_1 \times D_2 \times \cdots \times D_n$$

$D_1, D_2, \cdots, D_n$ sono domini di R, una relazione su n domini è detta avere **grado** n

Il numero di ennuple di una relazione è detto **cardinalità** della relazione, nelle applicazioni reali la cardinalità di una relazione è sempre finita.

Vediamo un esempio:

Dati i domini $D_1 = \{$corvo, gatto$\}$ e $D_2 = \{$bianco, nero, verde$\}$
Il prodotto cartesiano $D_1 \times D_2 =$
{(corvo, bianco), (corvo, nero), (corvo, verde), (gatto, bianco), (gatto, nero), (gatto, verde)}

1.2 Proprietà delle relazioni

Relazione con attributi

In una relazione di un database, ogni colonna ha un nome, quest'ultimo è detto **attributo** e rappresenta un particolare aspetto dei dati.

Una ennupla è una riga della tabella, rappresenta un insieme di valori, uno per ogni attributo.

Per ogni ennupla t e per ogni attributo A_k, possiamo indicare il valore che la ennupla assume su quell'attributo con $t[A_k]$

La rappresentazione tabellare

I valori di ciascuna colonna sono fra loro omogenei, i valori di un attributo appartengono allo stesso dominio.

Le righe sono diverse fra loro, una relazione non contiene mai ennuple identiche per via dell'orientamento ai valori.

L'ordinamento delle colonne è irrilevante poiché esse sono sempre identificate per nome e non per posizione, lo stesso vale per le righe in quanto identificate per contenuto.

L'orientamento ai valori

In un modello relazione, per via dell'orientamento ai valori, le righe di una tabella non hanno un identificativo fisso, l'unico modo per distinguere le righe tra di loro è guardare i valori che contiene.

Per fare riferimento ad una riga specifica non possiamo fare uso del numero della riga, in quanto queste non hanno un posizione fissa ma variabile nel tempo, quindi per andare a identificare una riga dobbiamo fare riferimento ai suoi valori interni.

Nel modello relazionale sono presenti due livelli principali per descrivere una relazione, cioè la tabella:

- Il livello intensionale rappresenta la struttura della tabella ed include il nome della relazione, più l'insieme degli attributi e i domini dei valori che ogni attributo può assumere.
- Il livello estensionale rappresenta l'insieme di ennuple che la tabella contiene in un dato momento, a differenza del livello intensionale questo può variare nel tempo.

2 L'informazione incompleta

2.1 Cause e soluzioni

Il modello relazione impone ai dati una struttura rigida, le informazioni sono rappresentate tramite ennuple con formati predefiniti, esso impone anche che tutti i valori vengano specificati ma è possibile che non tutti i valori siano noti o disponibili per cui è necessario trovare un modo per rappresentarli.

Possiamo farlo in due modi tramite l'utilizzo di valori del dominio "non utilizzati", come può essere la stringa vuota o il valore zero, però può capitare che non esistano valori non utilizzati o non significativi, per cui è stato introdotto il valore **NULL**, questo denota l'assenza di un valore del dominio.

Bisogna stare attenti perché il valore **NULL** non ha una semantica definita, a seconda dei casi può rappresentare un valore sconosciuto, un valore inesistente o un valore senza informazione.

3 Vincoli di integrità

3.1 Vincoli di integrità

Nelle basi di dati esistono istanze che pur sintatticamente corrette non rappresentano informazioni possibili per l'applicazione di interesse, come può essere un valore negativo associato alla popolazione di un paese.

Il vincolo di integrità indica delle proprietà che devono essere soddisfatte dalle istanze per far si che le informazioni siano corrette per l'applicazione, un vincolo quindi è una funzione booleana che associa ad ogni istanza i valori vero o falso.

Esistono due tipologie principali di vincoli:

- I vincoli intrarelazionali: vincoli sui valori o domini, vincoli di ennupla o vincoli di chiave
- I vincoli interrelazionali: vincoli di integrità referenziale, vincoli di chiave esterna

I vincoli di dominio

Un vincolo di dominio definisce l'insieme di valori validi che un attributo può assumere, la loro verifica si basa su predicati booleani.

Tali predicati sono valutati al momento dell'inserimento o modifica di un valore nella base di dati andando a garantire che vengano memorizzati solo se il predicato valutato è vero.

I vincoli di ennupla

I vincoli di ennupla sono condizioni da soddisfare sui valori di ciascuna ennupla, indipendentemente dalle altre ennuple, una possibile sintassi è un'espressione booleana che confronta i valori di un attributo con determinati valori scelti oppure tramite espressioni aritmetiche su di essi, anche essi sono verificati al momento dell'inserimento o della modificata.

I vincoli di dominio sono un caso particolare dei vincoli di ennupla che coinvolgono un solo attributo.

3.2 I vincoli di chiave

Una **chiave** è un insieme di attributi che identificano univocamente le ennuple di una relazione.

Una **superchiave** di una relazione è un insieme di attributi, può essere anche uno singolo, che permette di identificare in modo univoco ogni riga di una tabella, cioè se consideriamo una superchiave di una riga e la andiamo a confrontare con ogni riga della nostra tabella non ne possiamo trovare una uguale.

Quindi possiamo dire che una chiave è una superchiave con il minor numero possibile di attributi, tale che rimuovendo anche solo un attributo si perderebbe la proprietà di unicità, questo viene detto **minimalità**.

Trovare una superchiave minimale è il nostro obbiettivo all'interno delle basi di dati.

Nelle basi di dati ogni relazione ha almeno una chiave, per cui una relazione non può contenere ennuple uguali ed ogni relazione ha come superchiave l'insieme degli attributi su cui è definita.

L'esistenza delle chiavi garantisce l'accessibilità a ciascun dato della base di dati e tramite l'utilizzo delle chiavi riusciamo ad organizzare i dati in relazioni diverse andando ad usare le chiavi primarie ed esterne.

Nelle chiavi la presenza di valori nulli deve essere limitata, altrimenti non è possibile identificare le ennuple e realizzare facilmente i riferimenti da altre relazioni, quindi vogliamo imporre a ogni relazione la presenza di almeno una chiave priva di valori nulli, in modo che ci sia sicuramente una combinazione di dati che consenta l'accesso univoco alle ennuple della base di dati.

La **chiave primaria** è una chiave su cui non sono ammessi i valori nulli, solitamente è evidenziata tramite una sottolineatura, in SQL vengono definite tramite il vincolo PRIMARY KEY.

3.3 L'integrità referenziale

Vincoli di integrità referenziale

Solitamente informazioni in relazioni diverse sono correlate attraverso valori comuni, per via dell'orientamento ai valori, in particolare tramite i valori delle chiavi, solitamente primarie.

Un vincolo di integrità relazione impedisce l'inserimento di un valore in una chiave esterna se non esiste nella tabella referenziata, una chiave esterna, detta foreign key, indica un valore di una tabella che esiste

sicuramente in una tabella correlata, solitamente è associato alla chiave primaria ma basta che sia associato ad una chiave unica, in SQL vengono definite tramite il vincolo FOREIGN KEY.

4 Forme di relazione

Anomalie dovute alla cattiva strutturazione dei dati

L'introduzione dei vincoli di integrità garantisce la correttezza e buona qualità dei dati, ma non risolve tutte le anomalie possibili, quali eccesso di valori ridondanti e/o valori nulli nelle ennuple, possibilità di generare ennuple spurie.

Questo genere di anomalie conduce frequentemente a errori e ulteriori anomalie nelle operazioni di inserimento, aggiornamento e cancellazione dei dati.

Possiamo risolvere queste anomalie seguendo questi principi:

- un singolo oggetto reale deve corrispondere a una e una sola ennupla
- una classe di oggetti con le stesse proprietà deve corrispondere a una sola relazione
- ogni cella di una tabella deve contenere un valore atomico
- relazionare in modo corretto i dati di tabelle diverse in modo da evitare di generare corrispondenze scorrette in fase di join

Dipendenze funzionali

Una dipendenza funzionale tra due insiemi di attributi x, y, denotata come $x \to y$, indica che, in una relazione, se due tuple condividono gli stessi valori per gli attributi in x, allora devono avere gli stessi valori per gli attributi in y, cioè i valori di x determinano univocamente i valori di y.

Le dipendenze funzionali sono vincoli che descrivono come sono collegati tra loro i dati in una tabella, devono essere definite dal progettista sulla base della conoscenza della realtà di interesse, e servono a definire gli stati validi di una relazione, ovvero specificare legami logici fra dati che devono valere sempre, per qualsiasi istanza della relazione.

Un esempio è l'attributo matricola per uno studente universitario, qualsiasi studente è dotato di una matricola univoca.

Regole di inferenze per dipendenze funzionali

Un progettista individua le dipendenze funzionali più evidenti, dato l'insieme F delle dipendenze iniziali individuate, è possibile dedurre altre dipendenze sulla base delle seguenti regole di inferenza. L'insieme F^+ delle dipendenze funzionali individuate dal progettista e quelle inferite prende il nome di **chiusura** di F.

1. Regola *riflessiva*: se $X \supseteq Y$, allora $X \to Y$
2. Regola di *arricchimento*: $\{X \to Y\} \vDash XZ \to YZ$
3. Regola *transitiva*: $\{X \to Y, Y \to Z\} \vDash X \to Z$
4. Regola di *decomposizione*: $\{X \to YZ\} \vDash X \to Y$
5. Regola di *unione*: $\{X \to Y, X \to Z\} \vDash X \to YZ$
6. Regola *pseudo-transitiva*: $\{X \to Y, WY \to Z\} \vDash WX \to Z$

Normalizzazione delle relazioni

Nei nostri database vogliamo ottenere uno schema, cioè un insieme di tabelle collegate, che ha un certo livello di normalizzazione, in base a quanto vogliamo che sia pulito ed efficiente. Per fare ciò ci basiamo sulle dipendenze funzionali per controllare se una tabella con le sue chiavi è ben costruita, per fare questo controllo ci serviamo delle forme normali, cioè regole di qualità che una tabella dovrebbe rispettare. Se una tabella non rispetta una forma normale, significa che ha delle anomalie o soffre di ridondanza.

Gli obiettivi della normalizzazione sono di garantire un join senza perdita di dati, cioè quando spezziamo una tabella in più parti, dovremmo poter ricostruire la tabella iniziale solo tramite operazioni di join, non

deve comparire nessuna nuova riga o sparirne alcuna. Un altro obiettivo è la conservazione delle dipendenze, anche dopo aver spaccato la tabella, le dipendenze funzionali stabilite devono ancora essere valide.

Struttura degli attributi

- **Attributo semplice**: costituito da valori atomici
- **Attributo multivalore**: in cui un possibile valore è un insieme di valori
- **Attributo strutturato**: in cui un possibile valore è una ennupla di valori

4.1 Prima forma normale 1NF

Una relazione è in **prima forma normale 1NF** se ogni colonna contiene un solo valore, non ci sono gruppi ripetuti di dati dentro una stessa riga, i valori sono atomici.

Nel modello relazione la prima forma normale deve sempre essere rispettata, avendo dati semplici e atomici è facile:

- capire cosa significano
- gestire operazioni come inserimenti, modifiche e cancellazioni
- interpretare i risultati delle ricerche o query

Quindi in presenza di attributi strutturati o multivalore occorre sostituire l'attributo strutturato con attributi atomici e gli attributi multivalore con una relazione separata che contenga la chiave primaria della relazione originaria.

Vediamo un esempio in merito:

ID	**Nome**	**Indirizzo**	**Numero**
123	Marco Verdi	Via Roma 10, 00100 Roma	3329538439, 3310658391
124	Laura Bianchi	Via Milano 20, 20100 Milano	3849238425

Questa relazione non è normalizzata in quanto nome e indirizzo sono attributi strutturati, mentre numero è un attributo multivalore, quindi andrà suddiviso in due relazioni

ID	**Nome**	**Cognome**	**Via**	**Civico**	**CAP**	**Città**
123	Marco	Verdi	Via Roma	10	00100	Roma
124	Laura	Bianchi	Via Milano	20	20100	Milano

ID	**Numero**
123	3329538439
123	3310658391
124	3849238425

4.2 Seconda forma normale 2NF

La **seconda forma normale 2NF** si basa sul concetto di dipendenza funzionale completa, cioè una dipendenza funzionale $X \to Y$ si dice completa se tutti gli attributi di X sono necessari per determinare Y, se togli anche solo un pezzo di X, la dipendenza non funziona più.

Quindi se basta solo una parte di X per determinare Y abbiamo una dipendenza parziale e quindi non è adatta per la seconda forma normale.

Uno schema è in seconda forma normale se:

- è gia in prima forma normale
- ogni attributo non-primo, cioè che non fa parte di nessuna chiave, dipende dall'intera chiave primaria, non solo da una sua parte

In caso di una dipendenza parziale, è necessario effettuare la normalizzazione per avere una seconda forma normale valida, per normalizzare dobbiamo:
Spezzare la tabella, creare una nuova tabella per ogni dipendenza parziale, e in questa nuova tabella inserire la parte di chiave da cui dipende e il dato che dipende.

Vediamo un esempio in merito:

IDordine	**IDprod**	**NomeProd**	**PrezzoUnit**	**Quantità**	**nome**	**Cognome**
O123	P071	Monitor	200	2	Laura	Bianchi
O123	P030	Cuffie	100	1	Laura	Bianchi
O124	P043	Mouse	50	3	Mario	Rossi

Questa relazione è 1NF, per renderla in 2NF dobbiamo eliminare le dipendenze parziali presenti sulle chiavi primarie IDordine e IDprod, andando ad suddividere esse in nuove relazioni

IDOrdine	**IDProd**	**Quantità**
O123	P071	2
O123	P030	1
O124	P043	3

IDOrdine	**Nome**	**Cognome**
O123	Mario	Rossi
O124	Laura	Bianchi

ID	**Numero**	**PrezzoUnit**
P071	Monitor	200
P030	Cuffie	100
P043	Mouse	50

4.3 Terza forma normale 3NF

La **terza forma normale 3NF** si basa sul concetto di dipendenza transitiva. Una dipendenza transitiva succede quando tre attributi rendono valida la regola di transitività.

Uno schema di relazione è in terza forma normale se:

- è gia in seconda forma normale
- nessun attributo non-primo, cioè che non fa parte della chiave primaria, dipende dalla chiave tramite un altro attributo
- tutti gli attributi non-chiave devono dipendere direttamente dalla chiave, non passando da altri attributi

In caso di dipendenza transitiva bisogna spezzare la tabella, creare una nuova tabella che contiene l'attributo intermedio e gli attributi che lui determina, nella tabella origilinale tieni l'attributo intermedio come ponte di collegamento fra le nuove relazioni.

Vediamo un esempio in merito:

Iso3	**Valuta**	**Popolazione**	**Continente**	**PopolazioneArea**
ITA	Euro	60,793,753	Europa	742,453,000
DEU	Euro	81,203,521	Europa	742,453,000
AUS	DollaroAus	23,323,843	Oceania	37,391,000

In questo caso possiamo vedere le seguenti dipendenze transitive:

- Iso3 → Valuta
- Iso3 → Popolazione
- Iso3 → Continente, PopolazioneArea
- Continente → PopolazioneArea

Quindi possiamo suddividerle nelle seguenti due tabelle:

Iso3	**Valuta**	**Popolazione**	**Continente**
ITA	Euro	60,793,753	Europa
DEU	Euro	81,203,521	Europa
AUS	DollaroAus	23,323,843	Oceania

Continente	**PopolazioneArea**
Europa	742,453,000
Oceania	37,391,000

4.4 Forma normale di Boyce-Codd BCNF

La **forma normale di Boyce-Codd BCNF** è una versione ancora più rigorosa della terza forma normale, quindi una tabella BCNF sarà anche in terza forma normale, ma non vale il contrario.

Uno schema è in BCNF se ogni dipendenza funzionale $X \to Y$ che esiste nella tabella ha come X una superchiave, cioè se un insieme di attributi X determina un altro attributo Y, allora X deve essere abbastanza forte da identificare univocamente una riga, in pratica X deve essere o contenere una chiave.

Vediamo un esempio in merito:

Country	**Company**	**Manager**
ITA	Bayer	Saris
GRC	Bayer	Robertson
DEU	Bayer	Robertson
ITA	Bracco	Rossi
FRA	Bracco	Rossi

La sede locale di una azienda determina il dirigente di riferimento, un dirigente è legato a una sola azienda, ma può essere responsabile di diverse sedi nazionali, quindi si hanno le seguenti dipendenze funzionali:

- Country, Company → Manager
- Manager → Company

Country, Company è una chiave candidata, ma Manager non lo è, quindi abbiamo una relazione 3NF.
Per renderla in forma BCNF, possiamo fare tre diverse decomposizioni:

- R1(Country, Manager), R2(Country, Company), tutti gli attributi di R1 e R2 formano chiavi primarie
- R1(Manager, Company), R2(Company, Country), R1(Manager) e tutta R2 sono chiavi primarie
- R1(Manager, Company), R2(Manager, Country), R1(Manager) e tutta R2 sono chiavi primarie

Vediamo nel dettaglio la terza soluzione, che è quella corretta, in quanto nelle altre due se andiamo a ricostruire la relazione usando l'attributo comune, otterremo una tabella iniziale differente, cioè avremo una decomposizione additiva, che non è ammessa per avere una corretta forma normale.

Manager	**Company**
Saris	Bayer
Robertson	Bayer
Rossi	Bracco

Country	**Manager**
ITA	Saris
GRC	Robertson
DEU	Robertson
ITA	Rossi
FRA	Rossi

Andando a ricostruire la relazione usando l'attributo comune Manager, otteniamo la tabella di partenza.

5 Proprietà delle decomposizioni

5.1 Proprietà

Nella progettazione di un database possiamo usare la teoria delle normalizzazione per pensare al database come prodotto di un'unica grande relazione universale R che contiene tutti gli attributi dello schema. Da questa relazione iniziale andiamo ad ottenere uno schema composto da più relazioni tramite le decomposizioni.
Lo schema finale è detto decomposizione di R e denotato $D = R_1, R_2, \cdots, R_m$.
Ogni attributo di R deve essere presente in almeno una relazione R_i di D per conservare tutti gli attributi, in modo che vale:

$$\bigcup_{i=1}^{m} R_i = R$$

Una buona decomposizione dovrebbe rispettare alcune proprietà:

- **Conservazione delle dipendenze**: dato un insieme di dipendenze F su R, andiamo a definire **proiezione** di F su R_i$ l'insieme delle dipendenze $X \to Y$ di F^+ tali che gli attributi $X \cup Y$ siano tutti contenuti in R_i. Una decomposizione D conserva le dipendenze di R se:

  $$(\pi R_1(F) \cup \cdots \cup \pi R_m(F))^+ = F^+$$

  Cioè le regole devono rimanere valide anche dopo aver diviso la tabella.
- **Proprietà di join non-additivo (senza perdita)**: una decomposizione D è senza perdita rispetto all'insieme F di dipedenze di R se, per ogni stato di relazione r di R che soddisfa F, data l'operazione di JOIN $*$ vale che:

  $$*(\pi R_1(r), \cdots, \pi R_m(r)) = r$$

  Cioè riusciamo a ricostruire la tabella iniziale, senza errori sui dati.

5.2 Algoritmi di progettazione

Sintesi relazione in 3NF con conservazione delle dipendenze

Abbiamo come obiettivo la costruzione di un insieme di tabelle che siano in terza forma normale e mantenga tutte le dipendenze originali. il nostro input è una relazione universale R e un insieme di dipendenze funzionali F.

I passi da seguire sono:

- Semplificare F togliendo le dipendenze o attributi ridondanti nelle dipendenze, in modo da trovare la copertura minimale G di F.
- Creare una relazione che contiene X insieme agli attributi che X determina, se da X derivano più attributi, li metti tutti nella stessa relazione, nella quale avremo le dipendenze in G con X come parte sinistra.
- Mettere tutti gli attributi rimanenti in un'unica relazione per assicurare la conservazione degli attributi.

Decomposizione in BCNF senza perdita

Abbiamo come obiettivo di rendere lo schema ancora più pulito, come input abbiamo la relazione universale R e l'insieme di dipendenze funzionali F.

I passi da seguire sono:

- inizializzazione $D = R$
- ripetere le seguenti finche esiste una tabella Q in D che non è in BCNF
- trovare una dipendenza $X \to Y$ in Q che viola la BCNF
- sostituire Q con due nuove tabelle; una contenente gli attributi $X \cup Y$ e l'altra quelli $Q - Y$

Lezione 3 - Modellazione concettuale

1 Sistemi di basi di dati

1.1 Criteri di progettazione

Raccolta e analisi dei requisiti

Per primo passo è necessaria la comprensione degli obiettivi della base di dati, anche tramite una interazione con gli utenti che descrivono le necessità, i problemi, gli obiettivi facendo uso del linguaggio naturale, ideare una formulazione di descrizioni informali dei dati, cioè i loro requisiti, e delle operazioni che verranno effettuate sui dati, cioè requisiti funzionali.

Progettazione della base di dati

Progettazione concettuale: utilizzo di uno schema concettuale per rappresentare i requisiti informali sulla realtà di interesse tramite una descrizione concettuale formale e completa. Questo schema è indipendente dallo specifico DBMS che verrà utilizzato per la gestione della base di dati.

Progettazione logica: traduzione dello schema concettuale nel modello dei dati adottato dal DBMS scelto per la gestione della base di dati. Questo tipo di progettazione è dipendente dal DBMS scelto.

Progettazione fisica: la schema logico ottenuto viene completato con la specifica dei parametri fisici di memorizzazione dei dati, quindi l'organizzazione dei file e indici.

Modelli dei dati

Modelli concettuali: utilizzati nella fase di progettazione concettuale per costruire schemi concettuali, ovvero descrizioni dei dati ad alto livello di astrazione, indipendente dallo specifico DBMS. Un esempio di esso è il modello entità-relazione ER.

Modelli logici: utilizzati nella fase di progettazione logica per ottenere schemi logici, ovvero descrizioni dei dati processabili dal DBMS. Un esempio è il modello relazionale.

La nozione di astrazione

Il criterio fondamentale per la progettazione è l'**astrazione**, ovvero il processo mentale per estrapolare le proprietà e caratteristiche essenziali di un insieme di oggetti, ignorando le differenze e i dettagli non essenziali, ovvero una selezione delle proprietà rilevanti di un insieme di oggetti.

L'obbiettivo dell'astrazione è le definizione di descrizioni generali di classi di oggetti simili, prescrindendo dalle caratteristiche specifiche di ogni oggetto.

Criteri di astrazione

- **Classificazione**: individuazione di una classe o insieme che possa caratterizzare un gruppo omogeneo di individui specifici. Per esempio; simone, adriano, kevin, ... → Studente.
- **Aggregazione**: individuazione di un concetto che possa essere definito dall'insieme delle sue parti. Per esempio; Docente, Aula e Studenti → Corso
- **Generalizzazione**: individuazione di una classe che possa rappresentare entità simili tramite una entità generale. Per esempio; Studente, Docente → Persona

Interpretazione logica dei meccanismi di astrazione

I meccanismi di astrazione hanno un immediato corrispettivo in termini insiemistici, eccetto l'aggregazione che corrisponde alla relazione "essere parte di".

La classificazione è rappresentata dal simbolo di appartenenza $\in$, per esempio se x è classificato in una classe Y si dice $x \in Y$

La generalizzazione è rappresentata dal simbolo di sottoinsieme $\subseteq$, per esempio date due classi X e Y, possiamo dire che X generalizza Y, $X \subseteq Y$, ovvero se $\forall x | x \in X \rightarrow x \in Y$

La definizione di generalizzazione implica che tutte le caratteristiche proprie degli oggetti classificati in Y saranno necessariamente proprie anche degli oggetti classificati in X, possiamo dire che X eredita le caratteristiche di Y.

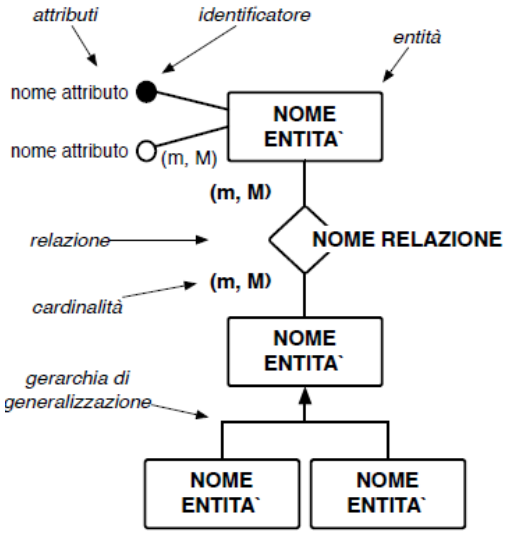
2 Progettazione concettuale

2.1 Il modello ER

Modello Entity-Relationship (ER)

Il principale modello dei dati usato per la progettazione concettuale è il modello ER. Ne esistono anche versioni estese o diverse per notazione, cosi come esistono anche altre alternative al modello ER per la progettazione concettuale, come UML.

Costrutti



I costrutti principali del modello ER sono:

- Entità
- Relazione
- Attributo

I costrutti secondari sono:

- Cardinalità
- Identificatori
- Attributi composti
- Gerarchie di generalizzazione

2.2 Entità, relazioni e attributi

Entità

L'entità rappresenta una classe di oggetti che possiedono proprietà comuni ed esistenza autonoma ai fini dell'applicazione di interesse.
Possono essere oggetti con esistenza fisica, come per esempio Territory, oppure oggetti che sistono a livello concettuale, come per esempio Name.

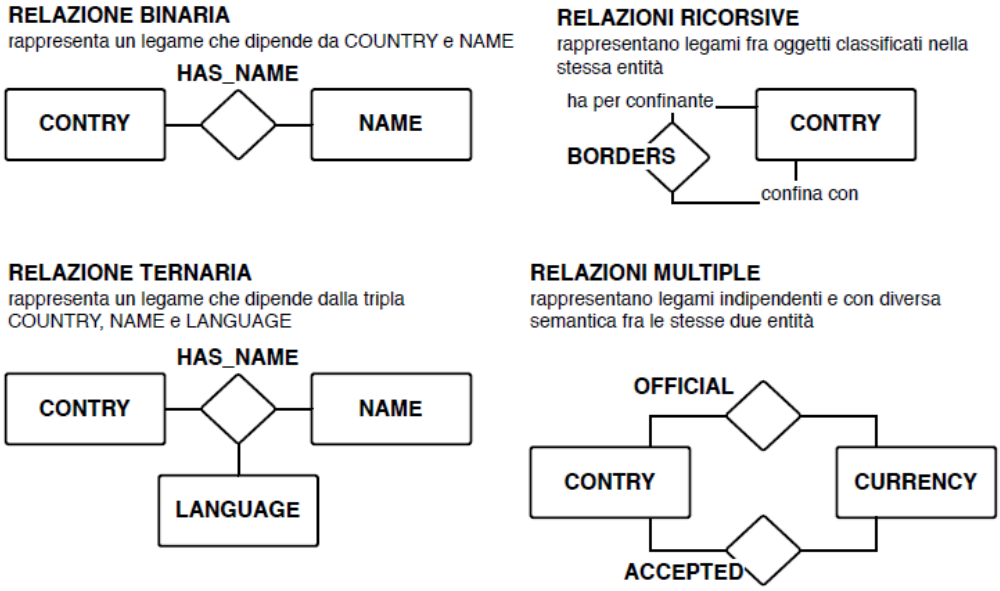
Ogni entità all'interno di uno schema ha un nome che la identifica univocamente ed è rappresentato graficamente da un rettangolo.

Relazione o associazione

La relazione rappresenta un legame logico tra due o più entità, per esempio l'associazione *has_name* fra country e name rappresenta il fatto che una nazione può avere diversi nomi in lingue diverse.

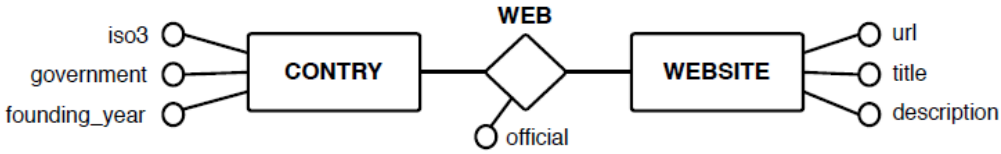
Un'associazione R all'interno di uno schema ER ha un nome che la identifica univocamente, essa è rappresentata da un rombo che collega le entità interessate.

Vediamo degli esempi di relazioni:

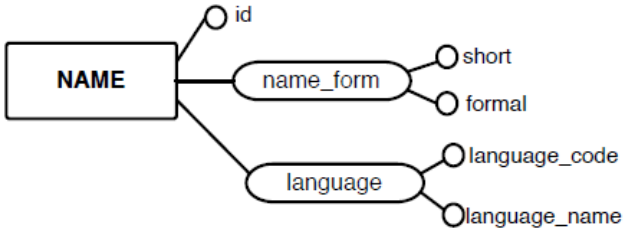


Attributi

Gli attributi sono necessari per descrivere una proprietà elementare delle entità e delle associazioni di interesse. Ogni istanza di entità o relazione possiede un valore per ciascuno dei suoi attributi. Il valore di un attributo appartiente a un dominio dell'attributo che contiene i valori ammissibili, come per esempio intero, stringa, data eccetera.



Esistono gli **attributi composti**, i quali raggruppano attributi che hanno affinità d'uso o di significato. L'insieme di questi attributi prende il nome di attributo composto.



2.3 Cardinalità

Vincoli di cardinalità

I vincoli di cardinalità esprimono le regole che stabiliscono la partecipazione delle istanze di un'entita a una relazione con altre entità. Data una relazione R tra due entità, E1 ed E2, avremo due vincoli di cardinalità differenti quello che va da E1 a R e quello da E2 a R.

Ogni vincolo di cardinalità è rappresentato da una coppia di valori (x,y), dove x indica la cardinalità minima e y la cardinalità massima.

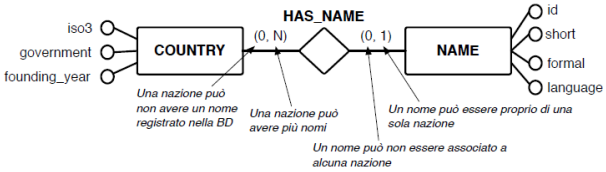
Cardinalità minima

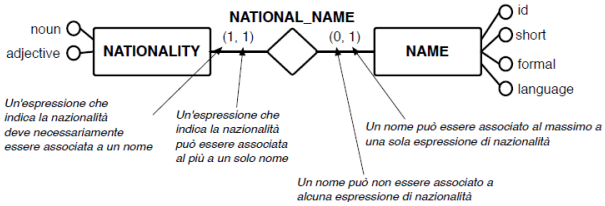
La cardinalità minima esprime il numero minimo di partecipazioni che una entità deve avere in una relazione, in pratica definisce se la partecipazione è obbligatoria o facoltativa, Se x = 0 allora la partecipazione è opzionale, mentre se x $\geq$ 1 la partecipazione è obbligatoria.

Cardinalità massima

La cardinalità massima esprime il numero massimo di partecipazioni che una entità può avere in una relazione, quindi serve a limitare il numero di coinvolgimenti.

Vediamo degli esempi relativi alle cardinalità:





Tipologie di associazioni binarie

In base ai valori delle cardinalità si hanno le seguenti tipologie di relazioni binarie:

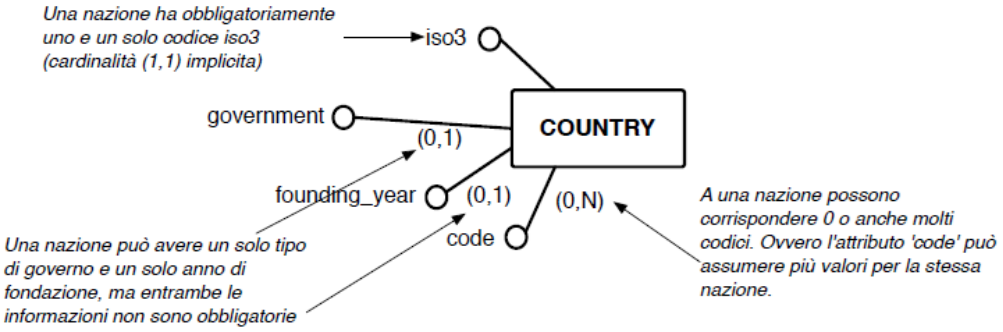
- (1,1) uno a uno; entrambe le entità hanno cardinalità 1
- (1,n) uno a molti; la cardinalità massima è n, la minima 1
- (n,m) molti a molti; entrambe le entità hanno cardinalità n,m

Cardinalità degli attributi

La cardinalità degli attributi rappresenta quanti valori può assumere un attributo per ogni singola istanza di entità o relazione. esso è rappresentato sempre da una coppia di valori.

La coppia (1,1) si verifica nella maggioranza dei casi, e di defualt è omesso, esso indica la obbligatorietà di uno e un solo valore.
Un valore di una coppia può essere nullo, solitamente la cardinalità minima può essere nulla ed indica che è un valore di attributo opzionale.
La cardinalità massima può essere n, ovvero quando un attributo può assumere diversi valori, fino ad un massimo di n.

Vediamo un esempio in merito:

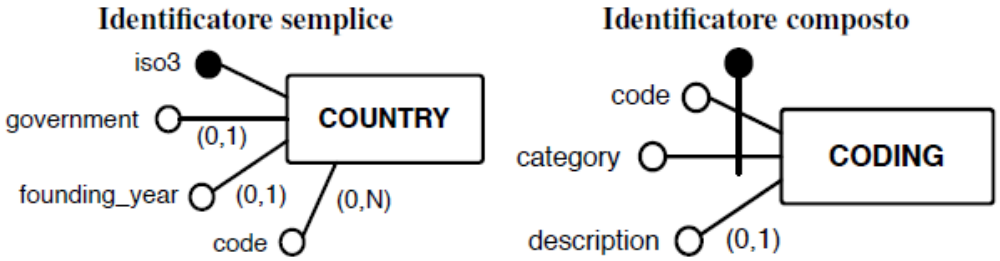


2.4 Identificatori

Un identificatore di una entità E è un insieme di attributi e/o entità connesse ad E che permettono di identificare univocamente le istanze di E. Vi sono due tipi di identificatori, quello interno e quello esterno.

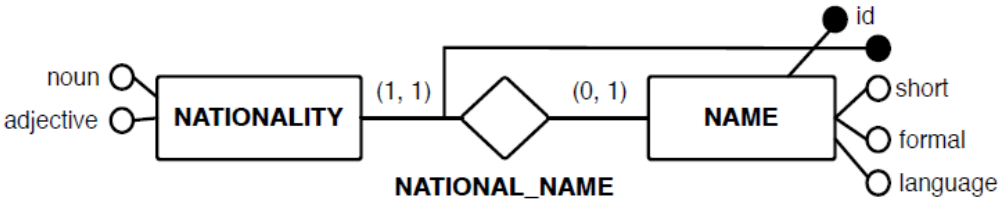
Gli identificatori interni

Un identificatore si dice interno quando è costituito esclusivamente da attributi dell'entità identificata. Può essere semplice, cioè costituito da un solo attributo oppure composto, cioè costituito da più attributi.



Gli identificatori esterni

Un identificatore esterno è necessario quando gli attributi di E non sono sufficienti per formare un identificatore per E, in questo caso E è detta **entità debole**. In questo caso si utilizzano entità con cui E ha un vincolo di dipendenza, per esempio si considerano tipi di associazioni binarie con cardinalità (1,1).



2.5 Gerarchie di generalizzazione

Una gerarchia di generalizzazione è quando un'entità E più generale, detta *superclasse*, viene specializzata in entità più specifiche $E1, E2, \cdots, En$ dette *sottoclassi*, che ereditano attributi e relazioni dalla superclasse.

Da questa definizione ne esce fuori il concetto di **ereditarietà**, infatti ogni proprietà di una superclasse E è ereditata da tutte le sottoclassi di E. Le proprietà ereditate non vanno rappresentate esplicitamente.

vincoli sulle specializzazioni

Una gerarchia è detta **totale** se ogni entità della superclasse è istanza di almeno una sottoclasse della specializzazione, altrimenti è detta **parziale**

Una gerarchia è detta **disgiunta** o **esclusiva** se un'istanza può essere istanza di una sola sottoclasse, altrimenti è detta **sovrapposta** quando può essere istanza di più sottoclassi contemporaneamente.

Esistono quindi le seguenti tipologie di gerarchie:

- Totale-Esclusiva (TE)
- Totale-Sovrapposta (TO)
- Parziale-Esclusiva (PE)
- Parziale-Sovrapposta (PO)

Interpretazione insiemistica

Data una gerarchia di generalizzazione fra un'entità padre E e le entità figlie E1, E2, ..., En

La gerarchia è totale se e solo se vale:

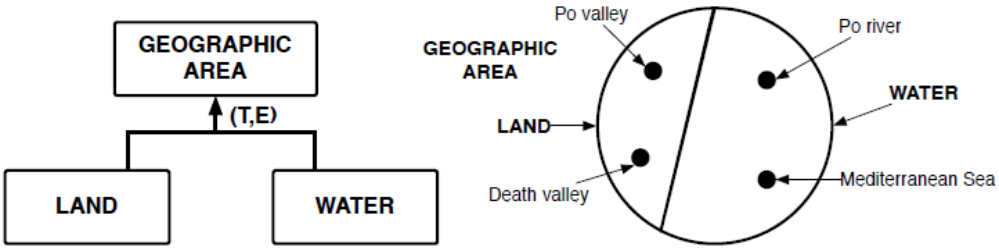
$$\bigcup_{i=1}^{n} E_i = E$$

La gerarchia è esclusiva se e solo se vale:

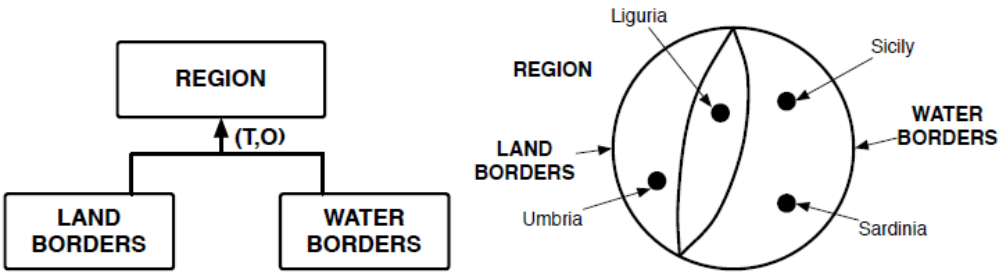
$$\bigcap_{i=1}^{n} E_i = \emptyset$$

Vediamo degli esempi in merito alle tipologie di gerarchie:

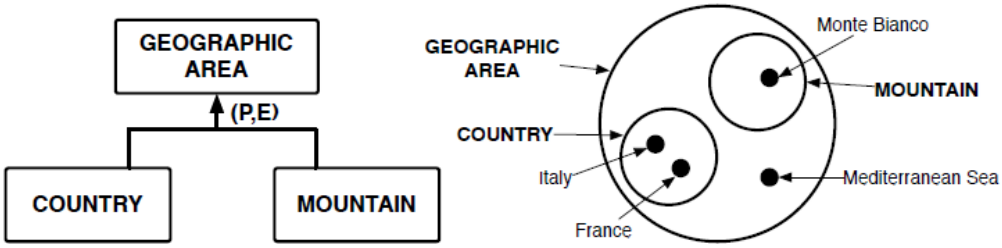
Totale e esclusiva



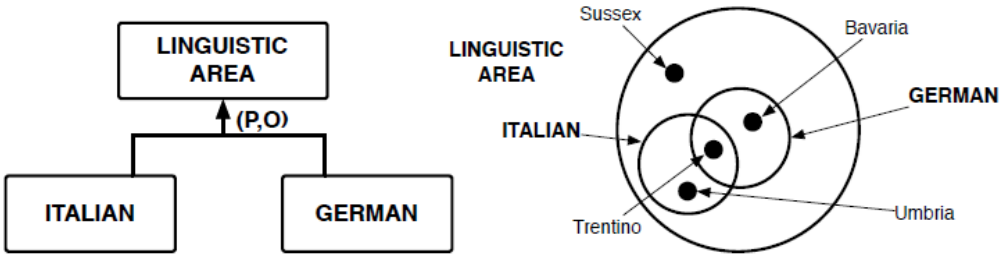
Totale e sovrapposta



Parziale e esclusiva



Parziale e sovrapposta



Lezione 4 – Progettazione Logica e SQL DDL

1 Progettazione logica

Fasi della progettazione logica

- **Ristrutturazione dello schema ER**: definizione di un nuovo schema ER andando a eliminare le gerarchie di generalizzazione e gli attributi composti e multivalore
- **Traduzione dello schema ER**: definizione dello schema relazionale, risultante dallo schema ER ristrutturato.

1.1 Ristrutturazione dello schema ER

Principi per la ristrutturazione dello schema ER

Le modifiche da apportare allo schema ER sono basare su considerazioni legate al carico della base di dati, cioè quanto e come verra utlizzata.

Le principali modifiche da tenere in considerazione sono l'analisi dei dati derivati (ridondanza), eliminazione delle gerarchie di generalizzazione e scelta degli identificatori primari.

Carico della base di dati

Il carico di un database è l'insieme di tutte le operazioni e applicazioni che il database dovrà supportare durante il suo utilizzo quotidiano. Per valutare quanto impegnativo sarà il carico, è necessario conoscere queste informazioni:

- Volume dei dati: il numero medio di istanze di ogni entità/associazione e il numero medio di partecipazioni delle entità alle associazioni
- Operazioni più frequenti (interrogazioni, inserimenti, aggiornamenti)
- Tipologia delle operazioni: se le operazioni sono interattive o batch, cioè in tempo reale o no
- Frequenza: il numero medio di esecuzioni delle operazioni in un certo intervallo di tempo
- Dati coinvolti: entità e/o associazioni interessate dalle operazioni

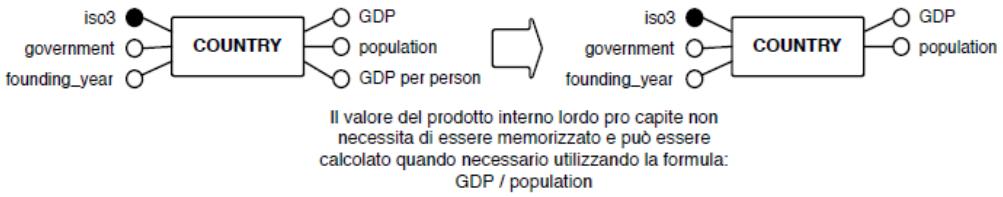
Dati derivati

Nei database ci possono essere attributi derivati, cioè informazioni che possono non essere memorizzate direttamente, ma che si possono calcolare partendo da altri dati già presenti. Possiamo scegliere di non salvare nel database questi dati ma andarli a calcolare solo quando servono.

I vantaggi sono la riduzione della quantità di dati memorizzati per via della eliminazione dell'attributo e nessuna necessità di mantenere la coerenza fra dati primari e derivati.

Lo svantaggio è dato da un maggiore carico di elaborazione per andare a calcolare il dato derivato.

Vediamo un esempio in merito:



Analisi dei dati derivati

L'eliminazione o il mantenimento di un dato derivato va valutata e documentata tentendo conto di alcuni fattori:

- In caso di eliminazione: il numero e la frequenza delle operazioni necessarie a calcolare il dato derivato

- In caso di mantenimento: il numero e la frequenza delle operazioni di aggiornamento necessarie a mantenere la persistenza e coerenza dei dati derivati
- Oltre a questi due fattori è necessario analizzare il risparmio/uso della memoria.

Eliminazione delle gerarchie di generalizzazione

Alcuni modelli logici, come il modello relazionale, non contemplano la nozione di gerarchia di generalizzazione. Quindi occorre sostituire le gerarchie con entità e associazioni, cercando di rispettare la semantica. Vi sono tre opzioni:

- opzione 1: mantenimento della sola entità padre
- opzione 2: mantenimento delle sole entità figlie
- opzione 3: mantenimento di tutte le entità

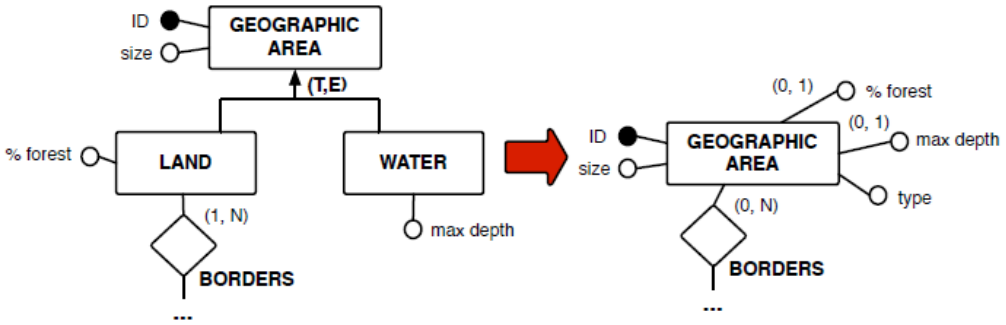
Opzione 1: mantenimento della sola entità padre

Questa strategia consiste nalla eliminazione delle entità figlie, cioè le sottoclassi, andando a conservare solo l'entità padre, per fare ciò bisogna seguire questi passi:

- Aggiunta di un nuovo attributo *tipo* sull'entità padre per memorizzare la sottoclasse di ogni istanza
- Aggiunta all'entità padre di tutti gli attributi e associazioni delle sottoclassi eliminate nell'entità padre
- Revisione delle cardinalità degli attributi aggiunti all'entità padre, in quanto alcuni potrebbero valere solo per alcune istanze, questi avranno cardinalità minima uguaòe a zero

Questi passaggi sono applicabili a qualsiasi tipo di gerarchia, sia totale che parziale.

Vediamo un esempio in merito:



L'attributo *type* può avere solo due valori e è obbligatorio, poiché la gerarchia è totale (ovvero un'area geografica è necessariamente o una terra o uno specchio d'acqua). Se la gerarchia fosse parziale, l'attributo *type* avrebbe cardinalità minima 0. In caso di gerarchie sovrapposte, *type* diventa un attributo multivalore.

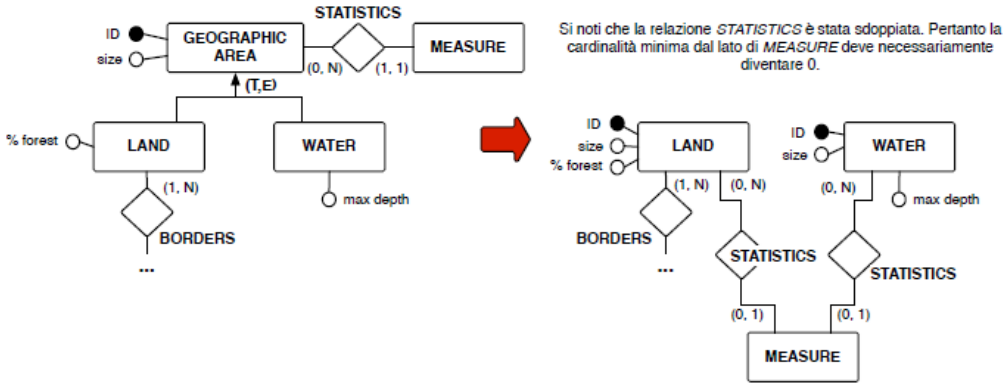
Opzione 2: mantenimento delle sole entità figlie

Questa strategia è l'opposto della precedente, si procede all'eliminazione dell'entità padre e si mantengono solo le entità figlie. per fare ciò bisogna seguire questi passi:

- Eliminazione dell'entità padre e trasferimento di tutti i suoi attributi e relazioni su tutte le entità figlie.
- Nessuna modifica delle cardinalità degli attributi e relazioni trasferite

Questi passaggi sono applicabili solo a gerarchie totali ed esclusive.

Vediamo un esempio in merito:



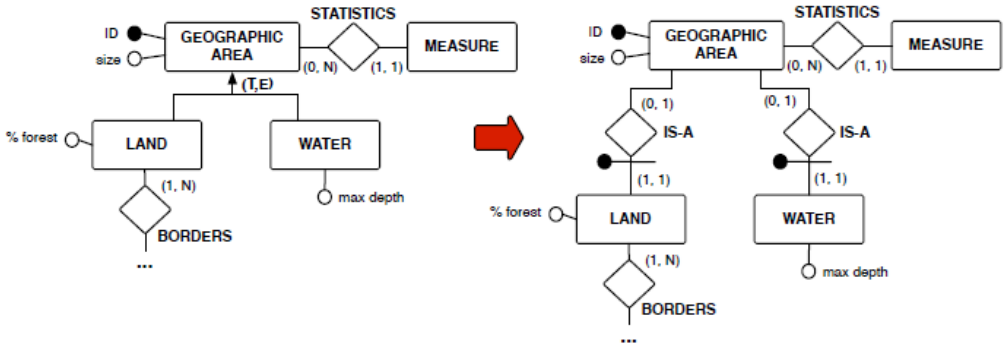
Opzione 3: mantenimento di tutte le entità

Questa strategia non va ad eliminare nulla, ma va a rappresentare la gerarchia in modo più esplicito attraverso associazioni uno a uno. I passaggi da seguire sono i seguenti:

- Creazione delle opportune associazioni 1:1 per rappresentare il legame *IS-A* espresso dalla gerarchia, ovvero che le entità figlie sono sottoinsiemi dell'entità padre e pertanto ogni loro istanza coincide con una sola istanza dell'entità padre
- Le entità figlie sono entità deboli rispetto all'entità padre, in quanto sono identificate tramite l'identificatore dell'entità padre

Questi passaggi sono applicabili a qualsiasi tipo di gerarchia.

Vediamo un esempio in merito:



Verifiche di correttezza

Dopo la modifica/eliminazione di una gerarchia, bisogna controllare che i dati restino coerenti, è necessario verificare che per ogni istanza delle specializzazioni, cioè le entità figlie, esista un'istanza dell'entità generica, ovvero il padre. In caso di generalizzazioni totali è necessario verificare che a ogni istanza dell'entità padre deve corrispondere un'istanza di qualche entità figlia.

Criteri di scelta della modalità di ristrutturazione

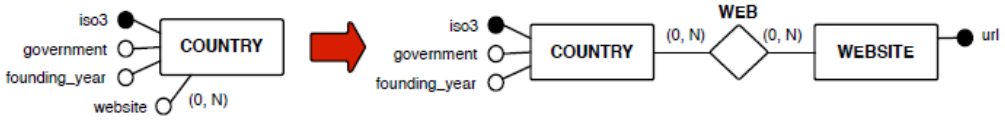
- **Opzione 1**: le operazioni usano principalmente gli attributi comuni a tutta la gerarchia, cioè quelli dell'entità padre. Questa opzione assicura un numero minore di accessi rispetto alle altre due alternative.
- **Opzione 2**: le operazioni usano insieme sia gli attributi generali che quelli specifici della sottoclassi, inoltre la gerarchia è totale ed esclusiva (T,E). Questa opzione assicura un risparmio di memoria, in quanto non ci sono attributi nulli come nella opzione 1 e ha meno accessi rispetto all'opzione 3 perchè non si visita l'entità padre per alcuni attributi.
- **Opzione 3**: le operazioni usano sia gli attributi dell'entità padre che quelli delle figlie, ma non necessariamente insieme. Questa opzione assicura un risparmio di memoria rispetto alla opzione 1, data

l'assenza di valori nulli e riduce i tempi di accesso rispetto all'opzione 2, in quanto ogni entità ha solo gli attributi che servono.

Eliminazione di attributi multivalore

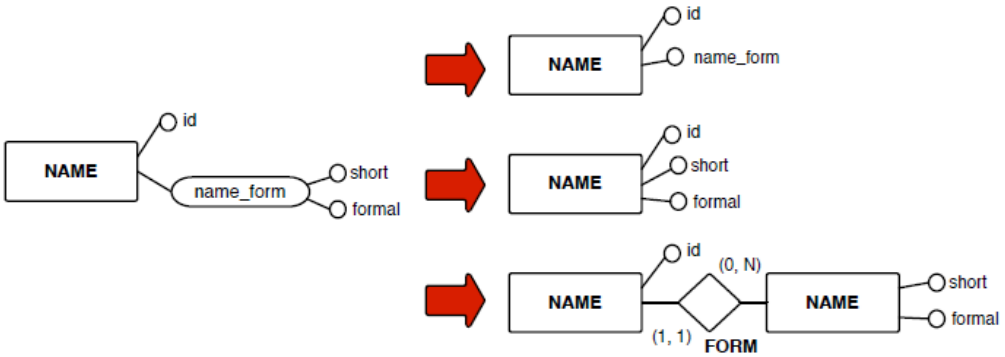
Per andare a rappresentare in modo più corretto questi attributi è necessario seguire questo procedimento:

- Creazione di una nuova entità per rappresentare l'attributo multivalore
- Creazione di una relazione tra l'entità originaria e la nuova entità, per collegare l'istanza principale con i suoi valori multipli
- La cardinalità della relazione è la stessa dell'attributo multivalore



Eliminazione di attributi composti

In caso di attributi composti, il modo più semplice e comune è andare a separarli in attributi semplici, un altra sceltà è la creazione di una nuova entità, se l'attributo composto ha significato proprio o presenta dipendenze transitive si crea una nuova entità e la si collega con una relazione 1:N all'entità originale.



1.2 Traduzione dello schema ER

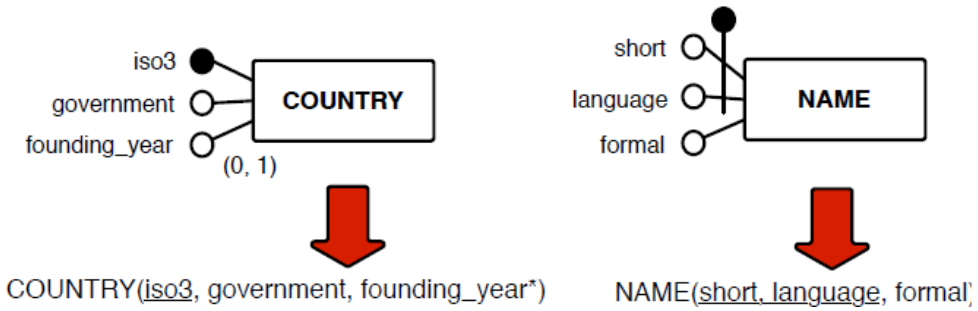
Metodologia generale

La traduzione dello schema ER ristrutturato allo schema relazione può essere vista come un processo composto da queste tre fasi principali:

1. Traduzione di entità
2. Traduzione di relazioni
3. Verifica di normalizzazione

Traduzione di entità

Data una entità $E = \{K, W\}$ con $K = \{a_1, \cdots, a_t\}$ come identificatore e $W = \{a_{(t+1)}, \cdots, a_{(t+n)}\}$ come l'insieme di attributi descrittivi. E è tradotto in una relazione $E(a_1, \cdots, a_t, a_{(t+1)}, \cdots, a_{(t+n)})$ in cui K è la chiave primaria. Gli attributi diventano le colonne della tabella.



Traduzione di relazioni/associazioni

La traduzione di associazioni dal modello ER ristrutturato al modello relazione permette diverse opzioni che dipendono dalla tipologia dell'associazione, come per esempio binaria o n-aria, o dalla tipologia di cardinalità, come 1:1, 1:N, N:M

Associazioni molti a molti (N:M) (ESEMPIO)

Data una associazione $R = \{C\}$ fra le entittà $E1 = \{K1, WI\}$ e $E2 = \{K2, W2\}$ in cui:

- Cardinalità massima di $(E1, R) = N$ e di $(E2, R) = M$
- C insieme di attributi della relazione R
- $K1$ e $K2$ sono identificatori di $E1$ e $E2$
- $W1$ e $W2$ sono attributi descrittivi di $E1$ e $E2$

Si traduce R come una relazione: $R(K1, K2, C)$
Nella quale C è presente solo nel caso in cui l'associazione R abbia attributi propri. La chiave primaria è composta da $K1 \cup K2$. Solo nel caso in cui si vogliano rendere possibili più relazioni fra le stesse istanze $E1$ e $E2$ al variare di C, si include anche C nella chiave primaria.

Associazioni uno a uno (1:1) (ESEMPIO)

Data una associazione $R = \{C\}$ fra le entittà $E1 = \{K1, WI\}$ e $E2 = \{K2, W2\}$ in cui:

- Cardinalità massima di $(E1, R) = 1$ e di $(E2, R) = 1$
- C insieme di attributi della relazione R
- $K1$ e $K2$ sono identificatori di $E1$ e $E2$
- $W1$ e $W2$ sono attributi descrittivi di $E1$ e $E2$

Si traduce la relazione R con una chiave esterna inserita in $E1$ o $E2$, quindi $E1(K1, W1, K2, C)E2(K2, W2)$ oppure $E1(K1, W1,)E2(K2, W2, K1, C)$

Notare che gli eventuali attributi C dell'associazione seguono la chiave esterna. La chiave esterna ammette valori nulli se la cardinalità minima dell'associazione è zero. In caso di entità deboli la chiave esterna è anche la chiave primaria.

Associazioni uno a molti (1:N) (ESEMPIO)

Data una associazione $R = \{C\}$ fra le entittà $E1 = \{K1, WI\}$ e $E2 = \{K2, W2\}$ in cui:

- Cardinalità massima di $(E1, R) = 1$ e di $(E2, R) = N$ o viceversa
- C insieme di attributi della relazione R
- $K1$ e $K2$ sono identificatori di $E1$ e $E2$
- $W1$ e $W2$ sono attributi descrittivi di $E1$ e $E2$

Si traduce la relazione R con una chiave esterna inserita nella relazione corrispondente all'entità con cardinalità massma 1: $E1(K1, W1, K2, C)E2(K2, W2)$ oppure si può creare una nuova relazione in cui però la chiave è composta dalla sola chiave esterna $R(K1, W1, K2, C)$

Notiamo che gli eventuali attributi C dell'associazione seguono la chiave esterna. La chiave esterna ammette valori nulli se la cardinalità minima dell'associazione è zero.

Altre tipologie di associazione (ESEMPI)

- **Associazioni n-arie**: si traducono con relazioni N:M con una chiave esterna per ogni entità implicata nell'associazione
- **Associazioni ricorsive**: si traducno come normali associazioni ma ridenominando le chiavi esterne in modo da evidenziarne il ruolo nell'associazione

1.3 Reverse engineering

Il reverse engineering è l'operazione di derivazione dello schema ER a partire dallo schema relazione, può essere utile quando occorre intervenire su un database pre-esistente e non documentato. Il criterio principale è partire dalle chiavi esterne per ricavare le associazioni e le cardinalità, nello schema ER non ci sono chiavi esterne.

2 Creazione dello schema relazione con SQL DDL

2.1 Introduzione a SQL

SQL è un linguaggio basato sulle query, ad oggi siamo arrivati alla sua terza versione, la quale aggiunge i trigger, i tipi composti, le viste ricorsive e il supporto per oggetti di grandi dimensioni.

2.2 Domini

Esistono i domini elementari, quali: numerici, caratteriali, temporali, questi ultimi sono suddivisi in vari sottotipi.

Domini numerici

I principali domini numerici si dividono in due categorie:

- Tipi numerici esatti, che rappresentano valori interi o decimali in virgola fissa, e sono: INTEGER, SMALLINT, NUMERIC, DECIMAL
- Tipi numerici approssimati, che rappresentano valori numerici approssimati in virgola mobile, e sono: REAL, DOUBLE, FLOAT

Stringhe e caratteri

Il principale dominio per indicare le stringhe è CHARACTER [VARYING] o abbreviato in CHAR, questo dominio permette di rappresentare un carattere o stringhe, di lunghezza fissa o variabile e definire una lunghezza massima predefinita, aggiungendo [(n)] nel comando.

Esistono altri domini di stringa quali: BIT, che rappresnta stringhe di bit, cioè di 0 e 1, e BOOLEAN, che rappresenta valori booleani di true e false.

Domini temporali

I principali domini temporali sono: DATE, rappresenta le date in formato YYYY-MM-DD, con le restrizioni necessarie, TIME che rappresenta i tempi nel formato HH:MM:SS e TIMESTAMP che è l'insieme dei due precedenti.

2.3 Creazione dello schema

La sintassi di base per la creazione di uno schema è la seguente:

```
CREATE SCHEMA NomeSchema
AUTHORIZATION IDAutorizzazione
[ElementoSchema1, ElementoSchema2, ...];
```

Nel quale *NomeSchema* rappresenta il nome dello schema, se mancante prende il nome dell'utente proprietario, mentre *IDAutorizzazione* è l'identificativo dell'utente proprietario dello schema, se mancante prende il valore di chi esegue il comando, infine ci sono gli elementi dello schema che sono i domini, le tabelle, le asserzioni, le viste e i privilegi.

Definizione di tabelle

La sintassi di base per la definizione di una tabella è la seguente:

```
CREATE TABLE NomeTabella (
NomeAttributo1 Dominio [DEFAULT Valore] [Vincoli],
NomeAttributo2 Dominio [DEFAULT Valore] [Vincoli],
...
[AltriVincoli]);
```

Vediamo un esempio in merito:

```
CREATE TABLE Ordini (
IDOrdine INT PRIMARY KEY,
DataOrdine DATE NOT NULL,
ClienteID INT NOT NULL,
Importo DECIMAL(10,2) DEFAULT 0.00,
CONSTRAINT fk_cliente FOREIGN KEY (ClienteID) REFERENCES Clienti(ID));
```

2.4 Vincoli

Vincoli intrarelazionali

Il vincolo è una proprietà che deve essere verificata da ogni istanza della basi di dati, per esempio NOT NULL indica che il valore NULL non è ammesso come valore dell'attributo, UNIQUE indica che il valore dell'attributo o un insieme di attributo deve essere unico, ciòe non può comparire su più righe.

Chiave primaria

La chiave primaria viene specificata tramite PRIMARY KEY, una volta sola per ogni tabella e può essere associata ad un attributo oppure su un gruppo di attributi, in questo caso viene detto chiave composta.

Vincoli inter-relazionali

Il vincolo di **integrità referenziale (foreign key)**, è un legame tra i valori dell'attributo della tabella corrente e i valori dell'attributo di un altra tabella. Impone che per ogni tupla della tabella che riferisce, il valore dell'attributo sia presente tra i valori di un attributo delle tuple della tabella riferita, eccetto il caso che la chiave esterna sia nullo.

Possiamo associare una *azione referenziale innescata* alle violazioni del vincolo di integrità referenziale generate da operazioni di modifica sulla tabella riferita, ovvero la cancellazione di tuple riferite da chiave esterna o modifiche del valore dell'attributo chiave riferito da chiave esterna. Queste azioni sono eseguite dal DBMS in caso di aggiornamenti sulla tabella riferita.

Le azioni possono essere le seguenti:

- CASCADE: i valori di chiave esterna della tabella che riferisce corrispondenti alla tupla modificata/cancellata nella tabella riferita vengono a loro volta modificati/cancellati
- SET NULL: i valori di chiave esterna della tabella che riferisce corrispondenti alla tupla modificata/cancellata nella tabella riferita sono posti a NULL
- SET DEFAULT: i valori di chiave esterna della tabella che riferisce corrispondenti alla tupla modificata/cancellata nella tabella riferita sono posti al valore predefinito
- NO ACTION: l'azione di aggiornamento/cancellazione sulla tabella riferita è rifiutata se vi sono tuple che fanno riferimento alla tupla da modificare/cancellare nella tabella che riferisce

Vincoli di dominio e di ennupla

Sono vincoli di integrità che impongono condizioni generiche sui valori delle tuple in una tabella, come i vincoli dipendenti dall'applicazione, i vincoli vengono specificati tramite l'uso di una o più clausole CHECK nel comando per la definizione di tabelle.
La condizione del vincolo (check-condition) è un qualsiasi predicato che può apparire nella clausola WHERE di una interrogazione SQL.

Definizione di domini

La sintassi di base per la definizione di un dominio è la seguente:

```
CREATE DOMAIN NomeDominio AS TipoDatoBase
[DEFAULT Valore]
[CONSTRAINT NomeVincolo CHECK (condizione)];
```

NomeDominio rappresenta il nome al nuovo dominio personalizzato, TipoDatoBase rappresenta il tipo di dato sul quale si basa il dominio, come INT, DATE, VARCHAR eccetera, il campo *DEFAULT* permette di associare un valore predefinito in caso non fosse riempito un attributo della tabella, *CONSTRAINT NomeVincolo CHECK condizione* indica un vincolo di integrita che limita i valori ammessi nel dominio.

2.5 Modifica dello schema

Abbiamo due principali comandi per la modifica dello schema, il comando ALTER e DROP.

il comando base di DROP è il seguente:

```
ALTER TABLE NomeTabella
[modifica specifica];
```

Le operazioni principali che possiamo fare con questo comando sono le seguenti:

Tabella 1: Operazioni con ALTER TABLE

Operazione	**Sintatti**
Aggiungere una colonna	ADD COLUMN NomeAttributo TipoDato
Eliminare una colonna	DROP COLUMN NomeAttributo
Impostare un valore di default	ALTER COLUMN NomeAttributo SET DEFAULT default
Rimuovere un valore di default	ALTER COLUMN NomeAttributo DROP DEFAULT
Aggiungere un vincolo	ADD CONSTRAINT NomeVincolo TipoVincolo (Colonne)
Rimuovere un vincolo	DROP CONSTRAINT NomeVincolo

il comando di base di DROP è il seguente:

```
DROP Tabella, Dominio, Vista eccetera; NomeAssociato [RESTRICT|CASCADE]
```

RESTRICT o CASCADE sono opzionali, il primo indica che il comando non è eseguito in presenta di elementi non vuoti mentre CASCADE elimina tutti gli elementi specificati, per esempio se eliminiamo un elemento che ne contiene altri implica la rimozione anche di questi ultimi.

Tabella 2: Operazioni con DROP

Operazione	**Sintatti**
Elimina una tabella e i suoi dati	DROP TABLE NomeTabella
Elimina una vista	DROP VIEW NomeVista
Rimuove un dominio personalizzato	DROP DOMAIN NomeDominio
Rimuove un vincolo da una tabella	ALTER TABLE NomeTabella DROP CONSTRAINT NomeVincolo
Elimina una colonna di una tabella	ALTER TABLE NomeTabella DROP COLUMN NomeAttributo
Elimina uno schema	DROP SCHEMA NomeSchema [CASCADE|RESTRICT]

Lezione 5 – L'algebra relazionale TO COMPLETE/REVIEW

L'**algebra relazionale** è il linguaggio procedurale per interrogare basi di dati relazionali. Il risultato di un'interrogazione è una relazione che può essere manipolata utilizzando gli operatori dell'algebra stessa.

Operatori dell'algebra relazione

- Gli operatori si suddividono in: *operatori insiemistici e operatori su relazioni*
- Operatori fondamentali: Unari(σ, π) e Binari($\cup$, $\times$, $-$)
- Operatori derivati: Binari($\bowtie$, $\bowtie_\theta$, $\cap$, $\div$)

Operatori insiemistici

- Due relazioni si dicono compatibili all'unione se sono dello stesso grado e sono definite sugli stessi attributi

Date due relazioni r e s compatibili all'unione, è possibile operare su esse con i seguenti operatori insiemistici:

- ***Unione***: l'unione $r \cup s$ è costituita dalle ennuple appartenenti a r o a s o ad entrambe
- ***Differenza***: la differenza $r - s$ è costituita dalle ennuple apparteneti a r e non a s
- ***Intersezione***: l'intersezione $r \cap s$ è costituita dalle ennuple appartenenti sia a r e sia a s

Compatibilità all'unione

$R(A_1, A_2, \dots, A_n)$ *e* $S(B_1, B_2, \dots, B_n)$ sono compatibili all'unione se:

1. R e S hanno lo stesso grado, cioè stesso numero di attributi
2. Domini corrispondenti degli attributi, gli attributi corrispondenti nelle due relazioni devono appartenere allo stesso dominio ($Dom(A_i) = Dom(B_i) \quad \forall i = 1, \dots, n$)

Operatori unari

Selezione (σ): Restituisce una relazione contenente l'insieme delle sole ennuple della relazione operando che soddisfano particolari condizioni, espresse mediante una formula proposizionale.

Proiezione (π): Restituisce come risultato una relazione contenente l'insieme delle ennuple della relazione operando limitate su particolari attributi.

Nella selezione si tratta di una **decomposizione orizzontale** in quanto va a selezionare alcune ennuple della relazione originale Nella proiezione si tratta di una **decomposizione verticale** in quanto va a selezionare alcuni insiemi di ennuple della relazione originale

Operatori binari

Prodotto cartesiano ($\times$)

Date due relazioni $r = \mathrm{R(X)}$ e $s = \mathrm{S(Y)}$ con $X \cap Y = \emptyset$ il prodotto cartesiano $r \times s$ è una nuova relazione definita sull'insieme di attributi XY contenente tutte le possibili combinazioni delle tuple di r e s.

Operatori Join

Theta-Join ($\bowtie_\theta$)

Date due relazioni $r = \mathrm{R(X)}$ e $s = \mathrm{S(Y)}$ con X e Y disgiunti, il Theta-Join $r \bowtie_\theta s$ è una relazione definita su XY composta dalle ennuple risultato della concatenazione di ennuple di r con ennuple di s che soddisfano le condizioni di confronto $A\theta B$ con $\mathrm{A} \in \mathrm{X}$ e $\mathrm{B} \in \mathrm{Y}$

- θ è una espressione composta per mezzo di operatori di confronto ($=, \neq, >, <, \geq, \leq$)
- Se il confronto è di uguaglianza, l'operazione è detta **Equijoin**
- Possiamo notare che il join $\mathrm{R} \bowtie_{A=B} \mathrm{S}$ è del tutto equivalente all'operazione $\sigma_{A=B}(\mathrm{R} \times \mathrm{S})$

Join naturale ($\bowtie$)

Date due relazioni r(YX) e s(XZ), il join naturale di r e s è l'operazione che combina le tuple delle due relazioni basandosi sull'uguaglianza dei valori dell'attributo X, il quale è comune ad entrambe.

Possiamo dare le seguenti definizioni:

- Due ennuple sono *joinabili* se è possibile effettuare l'operazione di join su di esse
- Una ennupla è detta *dangling* quando non contribuisce all'operazione di join
- Una operazione di join viene detta *completa* quando non contiene ennuple dangling
- Tramite una *ridenominazione*(ρ) possiamo rendere compatibili all'operazione di join due attribuiti uguali concentualmente ma con sintassi differente

Proprietà del join naturale

- Il join naturale è commutativo e associativo
- Può essere espresso in termini di:
  - Prodotto cartesiano
  - Selezione ennuple con valori uguali per attributi in comune
  - Proiezione di r e s eliminando duplicazioni
- Se r(X), s(Y) con X $\cap$ Y = $\emptyset$, allora $r \bowtie s = r \times s$
- Se r(X), s(Y) con X = Y, allora $r \bowtie s = r \cap s$

Outer Join

L'outer join è una variante del join che conserva anche le ennuple dangling (quelle che non trovano corrispondenza), inserendo valori NULL nei campi mancanti delle relazioni. Questo permette di evitare la perdita di dati che si verifica nei join naturali e theta join.

Tipologie di Outer Join:

1) Left outer join (Left Join)

- Mantiene tutte le ennuple della relazione di sinistra

2) Right Outer Join (Right Join)

- Mantiene tutte le ennuple della relazione di destra

3) Full Outer Join (Full Join)

- Mantiene tutte le ennuple di entrambi le relazioni

Divisione

La divisione si applica solo a relazioni r(Z) e s(X), dove X $\subseteq$ Z. La divisione va a restituire una relazione T(Y), dove Y = Z - X, contenente tutte le ennuple t tali che:

- Esistano ennuple tr in r con tr[Y] = t
- Per ogni ennupla ts in s, esista una ennupla tr in r con tr[X] = ts

Intuitivamente la divisione verifica una condizione o corrispondenza fra una o più ennuple di r e tutte le tuple di s.

Equivalenza di espressioni algebriche

Caratteristiche generali

-
-
-
-

regole di trasformazione

-
-
-
-
-

Lezione 6 – SQL

Introduzione

Le operazioni basilari per i database sono le DML, Data Manipulation Language, queste permettono di modificare i dati all'interno di una tabella.

L'operazione di **inserimento** di valori in una tabella di un database avviene tramite il seguente schema di comandi:

INSERT INTO *NomeTabella* (*ListaAttributi*) VALUES (*ListaDiValori*);

Un esempio pratico è il seguente:

```
INSERT INTO studenti (matricola, nome, età) VALUES (345216, 'Luigi', 21);
```

Da tenere presente che è anche possibile andare ad importare insiemi di righe di dati da altre tabelle del nostro database o anche in maniera estenstiva scrivendo le varie ennuple separate da parantesi rotonde all'interno del comando citato in precedenza.

L'operazione di **modifica** dei valori di una tabella di un database avviene tramite il seguente schema di comandi: UPDATE *NomeTabella* SET *Attributo* = (*Valore*)

Un esempio pratico è il seguente:

```
UPDATE studenti SET età = 20;
```

Prestiamo attenzione che il seguente comando lavora sull'intera tabella, se vogliamo andare a modificare in modo più selettivo dobbiamo fare l'utilizzo dell'opzione WHERE, per esempio:

```
UPDATE studenti SET età = 20 WHERE matricola = 345216;
```

L'operazione di **Cancellazione** di valori in una tabella di un database avviene tramite il seguente schema di comandi:

DELETE FROM *NomeTabella*

Un esempio pratico è il seguente:

```
DELETE FROM studenti;
```

Anche in questo caso il comando precedente lavora sull'intera tabella, quindi se vogliamo una cancellazione selettiva dobbiamo fare utilizzo del comando aggiuntivo WHERE, per esempio:

```
DELETE FROM studenti WHERE matricola = 345216;
```

Interrogazioni

Introduzione alle interrogazioni

Per effettuare le interrogazioni sui dati salvati nei nostri database avremo un linguaggio SQL dedicato detto DQL, Data Query Languange. il DML, come SQL, è un linguaggio dichiarativo cioè l'utente descrive l'obbiettivo dell'operazione ma non specifica i dettagli su come il database deve eseguire l'operazione. l'interprete del DBMS analizza l'istruzione della query e genera il corrispettivo codice in linguaggio procedurale, il tutto in modo nascosto dall'utente.

Il Query Optimizer è un modulo del DBMS che ottimizza la query trovando il modo più efficiente per eseguire l'interrogazione.

il linguaggio SQL può essere incluso anche in altri programmi/applicazioni, scritti anche con linguaggi differenti, per interagire con i nostri database.

L'istruzione SELECT

l'istruzione SELECT è il comando fondamentale del DQL, questo viene usato per recuperare dei dati da una o più tabelle.

La sintassi di base del SELECT è la seguente:

SELECT *ListaAttributi* FROM *ListaTabelle* (WHERE *Condizione*)

- SELECT: Indica gli attributi, colonne, dalle quali recuperare i dati cercati.
- FROM: Nome delle tabelle sulle quali la query verrà valutata.
- WHERE: Espressione booleana di ricerca che filtra le ennuple in base ad una condizione.

Esempi di utilizzo:

Selezione di una intera tabella:

```
SELECT * FROM clienti;
```

Selezione di colonne specifiche:

```
SELECT nome, cognome FROM clienti;
```

Selezione di una intera tabella o di colonne specifiche con filtrazione sui dati:

```
SELECT * FROM clienti WHERE città = 'roma';
SELECT nome, cognome FROM clienti WHERE città = 'roma';
```

La clausola WHERE

La clausola WHERE necessità di espressioni booleane costruite tramite l'utilizzo di operatori di confronto, che possono essere combinati tra di loro tramite gli operatori logici AND, OR, NOT.

Gli operatori di confronto basici disponibili in SQL sono i seguenti: $=, <>, <, >, <=, >=, IS NULL, IS NOT NULL. Inoltre sono presenti anche operatori di confronto avanzanti come possono essere BETWEEN, IN, EXISTS e LIKE.

Alcuni esempi di utilizzo:

```
SELECT * FROM clienti
WHERE Città = 'Roma';

SELECT nome, cognome FROM clienti
WHERE età > 18;
```

Se vogliamo andare a filtrare i risultati in base a pattern di testo possiamo fare uso dell'operatore LIKE, tramite "%" possiamo rappresentare una sequenza di caratteri, mentre tramite "_" un carattere singolo, per esempio:

```
SELECT nome, cognome FROM clienti
WHERE nome LIKE 'Mar%';
```

Questo esempio seleziona tutti i nomi che incominciano con "Mar".

```
SELECT nome, cognome FROM clienti
WHERE nome LIKE '_a___';
```

Questo esempio invece filtra a parole di 5 caratteri, dove la seconda lettera è il carattere 'a'

Possiamo andare ad eseguire la **ridenominazione degli attributi** ricavati dalla clausola WHERE, questo si effettua tramite l'operatore 'AS' che indicherà il nome della nuova colonna, contenente il dato cercato.

La ridenominazione permette di eseguire le operazioni matematiche di base (somma, moltiplicazione…) sui dati selezionati, ma anche operazioni più avanzate come l'arrotondamento, la potenza eccetera.

Vediamo un esempio di moltiplicazione e di potenza:

```
SELECT Nome, Età, Età * 12 AS Eta_in_Mesi
FROM Studenti
WHERE Età > 18;
```

```
SELECT Nome, Età, POWER(Età,2) AS EtaAlQuadrato
FROM Studenti
WHERE Età > 18;
```

Join

L'operatore Join ha la funzione di combinare righe da due o più tabelle sulla base di una condizione comune, solitamente tramite una chiave primaria ed una esterna.

I tipi principali di JOIN sono:

- Inner Join: Restituisce solo le righe che corrispondono in entrambe le tabelle
- Left Join: Tutte le righe della prima tabella, anche se non c'è corrispondenza
- Right Join: Tutte le righe della seconda tabella, anche se non c'è corrispondenza
- Full Join: Tutte le righe di entrambe le tabelle, combinate dove possibile

Esiste anche il natural join, nel quale non è specificata alcuna condizione, ma verrà effettuato un equijoin implicito di due colonne con lo stesso nome, condizione necessaria perchè funzioni, se non presenti è necessario effettuare una ridenominazione.

Vediamo degli esempi per ogni tipologia:

Inner Join:

```
SELECT m.id, m.official_title, imdb.rating.movie, imdb.rating.score
FROM imdb.movie m
INNER JOIN imdb.rating on m.id = rating.movie
WHERE imdb.rating.score > 7;
```

Left join:

```
SELECT m.id, m.official_title, imdb.rating.movie
FROM imdb.movie m
LEFT JOIN imdb.rating on m.id = rating.movie;
```

Right join:

```
SELECT m.id, m.official_title, imdb.rating.movie
FROM imdb.movie m
RIGHT JOIN imdb.rating on m.id = rating.movie;
```

Altre condizioni di selezione

Tramite il predicato **is null** possiamo selezionare le righe con valori nulli, esso può essere anche negato per non andarle a selezionare.

Tramite l'operatore **SELECT DISTINCT** possiamo andare ad eliminare eventuali righe uguali dal risultato di una interrogazione.

Tramite gli attributi *ASC* e *DESC* abbinati all'operatore **ORDER BY** possiamo eseguire l'ordinamento delle ennuple risultate da una interrogazione. teniamo presente che il valore NULL ha significato più altro, quindi in caso di ordinamento decrescente viene messo in cima.

Interrogazioni complesse

Operatori aggregati

Gli operatori aggregati permettono di calcolare valori riassuntivi su insiemi di righe, come somme, medie, conteggi eccetera. I principali operatori sono COUNT, SUM, AVG, MIN, MAX. l'operatore aggregato viene applicato al risultato di una interrogazione avvenuta tramite gli operati FROM e WHERE.

L'operatore COUNT

L'operazione COUNT (*) restituisce il numero di righe di una tabella specificata, COUNT('attributo') restituisce il numero di righe della tabella che possiedono valori diversi da null, a COUNT possiamo applicare l'attributo DISTINCT che conterà solamente i valori diversi all'interno di una tabella. vediamo alcuni esempi:

```
SELECT COUNT(*) AS TotaleStudenti
FROM Studenti;

SELECT COUNT(nome) AS TotaleStudenti
FROM Studenti;

SELECT COUNT(DISTINCT nome) AS TotaleStudenti
FROM Studenti;
```

Gli altri operatori

Gli operatori di MIN e MAX permettono di ottenere il valore minimo e massimo dei valori presenti in una determinata colonna, l'operatore SUM permette di sommare i vari valori della colonna specificata, mentre l'operatore AVG permette di calcolare la media dei valori della colonna specificata. vediamo degli esempi:

```
SELECT AVG(Età) AS EtaMedia
FROM Studenti;

SELECT SUM(Età) AS SommaEta
FROM Studenti;

SELECT MIN(Età) AS EtaMinima, MAX(Età) AS EtaMassima
FROM Studenti;
```

Il raggruppamento

Il raggruppamento ci permette di appplicare gli operatori aggregati a sottogruppi di tuple di una tabella in base al valore degli attributi, la sintassi base è la seguente:

SELECT colonna1, funzione_aggregata(colonna2) FROM tabella GROUP BY colonna1;

Vediamo un esempio in merito:

```
    SELECT Città, AVG(Età) AS EtaMedia
    FROM Studenti
    GROUP BY Città;
```

Teniamo a mente che ogni colonna nel SELECT deve essere una colonna nel GROUP BY oppure il risultato di funzione aggregata. Inoltre il GROUP BY viene sempre prima di HAVING e dopo WHERE, se presenti.

Ricordiamo che la clausola WHERE filtra le righe prima del raggruppamento, mentre la clausa HAVING che filtra dopo il raggruppamento, per esempio:

```
    SELECT Città, COUNT(*) AS NumeroStudenti
    FROM Studenti
    GROUP BY Città
    HAVING COUNT(*) > 1;
```

Interrogazioni di tipo insiemistico

Le operazioni insiemistiche presenti in SQL sono UNION, INTERSECT e EXCEPT, di default eseguono l'eliminazione dei duplicati, se vogliamo tenere questi ultimi dobbiamo specificarlo con l'aggiunta di ALL.

Vediamo degli esempi per ogni tipo di operazione:

```
Unione tra studenti non laureati e laureati
    SELECT Nome FROM Studenti
    UNION
    SELECT Nome FROM Laureati;

Selezione di studenti laureati
    SELECT Nome FROM Studenti
    INTERSECT
    SELECT Nome FROM Laureati;

Selezione di studenti non laureati
    SELECT Nome FROM Studenti
    EXCEPT
    SELECT Nome FROM Laureati;
```

Interrogazioni nidificate

Una interrogazione nidificata consiste in una query che al suo interno possiede una subquery, essa è una query interna scritta tra parentesi, che viene eseguita prima della query esterna e i cui risultati vengono usati da quest'ultima.

Esistono tre tipologie di subquery:

- subquery scalare: restituisce un solo valore
- subquery che restituisce una colonna: spesso usata con IN, ANY, ALL

- subquery correlata: fa riferimento a una colonna della query esterna

vediamo degli esempi, uno per tipologia:

```
Subquery scalare; trova il valore della popolazione con il PIL più alto

    SELECT value AS popolazione
    FROM Measure
    WHERE label = 'Population'
        AND country = (
          SELECT country
          FROM Measure
          WHERE label = 'GDP'
          ORDER BY value DESC
          LIMIT 1);

Subquery normale; trova il nome dei paesi monarchici che hanno dati di popolazione

    SELECT name
    FROM Country
    WHERE iso3 IN (
      SELECT country
      FROM Measure
      WHERE label = 'Population'
    )
      AND government LIKE '%monarchy%';

Subquery correlata; mostra i paesi e la loro popolazione solo se superiore
alla media globale

    SELECT name, (
        SELECT value
        FROM Measure
        WHERE label = 'Population' AND country = c.iso3
    ) AS popolazione
    FROM Country c
    WHERE (
        SELECT value
        FROM Measure
        WHERE label = 'Population' AND country = c.iso3
    ) > (
        SELECT AVG(value)
        FROM Measure
        WHERE label = 'Population');
```

Predicato EXISTS

Il predicato EXISTS(subquery) restituisce TRUE se la subquery restituisce almeno una tupla; altrimenti restituisce FALSE.

In SQL esiste anche NOT EXISTS che funziona al contrario.

vediamo un esempio:

```
Trovare le nazioni con lo stesso nome in lingue diverse
    SELECT country, gazeteer, script FROM name N

    WHERE EXISTS (
        SELECT * FROM name X
        WHERE N.gazeteer = X.gazeteer AND N.script <> X.script );
```

divisione

Le subquery correlate e l'operatore di NOT EXISTS sono necessari per implementare l'operazione di divisione in SQL.

La specifica della divisione in SQL fa uso dell'operatore NOT EXISTS e richiede di ragionare in base al concetto di controesempio; una persona A verifica l'interrogazione se non esiste una repubblica che A non abbia visitato, ovvero data una tupla A di persona non deve esistere una tupla C di country tale che C sia una repubblica e che non esista una relazione TRAVEL fra A e C.

vediamo un esempio SQL:

```
SELECT code, name, surname FROM person A
WHERE NOT EXISTS (
    SELECT * FROM country C
    WHERE government LIKE '%republic%'
    AND NOT EXISTS (
        SELECT * FROM travel T
        WHERE T.person=A.code AND T.country=C.iso3
));
```

Piano di esecuzione delle interrogazioni

Explain

Le interrogazioni SQL sono trasformare in una successione di operazioni atomiche sui dati denominata **piano di esecuzione**, è possibile accedere al piano di esecuzione attraverso la clausola EXPLAIN.

Vediamo un esempio in merito:

```
    EXPLAIN
    SELECT name
    FROM Country
    WHERE iso3 IN (
        SELECT country
        FROM Measure
        WHERE label = 'Population'
          AND value > (
        SELECT AVG(value)
        FROM Measure
        WHERE label = 'Population'
    )
);
```

Interrogazioni avanzate TO COMPLETE

La clausola WITH

Una Common Table Expression (CTE) è una tabella temporanea che esiste e può essere utilizzata solo nell'ambito di una interrogazione, le CTE sono realizzate attraverso la clausola WITH, tramite la seguente sintassi:

WITH [RECURSIVE] with query [, …]
SELECT …

In cui la with query 'e:
with query name [(column name [, …])] AS (SELECT …)

Vediamo il seguente esempio:

Interrogazioni ricorsive

In generale la clausola WITH è prevalentemente una facilitazione sintattica per interrogazioni che sarebbe comunque possibili attraverso query nidificate, correlate e viste. Aggiungendo la clausola RECURSIVE diventà però possibile usare WITH per interrogazioni ricorsive altrimenti impossibili in SQL.

Vediamo un esempio astratto che genera ricorsivamente numeri:

```
WITH RECURSIVE t(n) AS (
SELECT 1
UNION ALL
SELECT n+1 FROM t WHERE n < 100
)
SELECT n FROM t;
```

Una query ricorsiva è sempre composta da un termine di passo (non ricorsivo) seguito da UNION (ALL) e da un termine ricorsivo. Solo il termine ricorsivo può contenere un riferimento all'output della query.

La differenza tra UNION e UNION ALL è che il primo scarta i duplicati, mentre il secondo no.

L'esecuzione di una query ricorsiva inizia dal passo base, una volta eseguito questo si inizia l'esecuzione del termine ricorsivo finchè la condizione di chiusura non è soddisfatta. Le query ricorsive sono spesso utilizzare per interrogare tabelle che rappresentano relazioni gerarchiche o relazioni di connessioni fra oggetti.

vediamo degli esempi di interrogazioni ricorsive:

Viste TO COMPLETE

Uso di viste in interrogazioni SQL

Una vista in SQL è una tabella derivata a partire da altre tabelle, essa è una tabella virtuale, in quanto non ci sono tuple istanza della vista memorizzate nella base di dati. Anche se virtuale, può essere utilizzata nella formulazione di interrogazioni come se fosse memorizzata. Da tenere presente che possono anche esisistere viste materializzate e quindi salvate nella base di dati.

Generazione di viste

```
CREATE VIEW NomeVista [(ListaAttributi)] AS SelectSQL
[WITH [LOCAL | CASCADED] CHECK OPTION]
```

In generale le viste sono utili per:

- specificare una nuova tabella che frequentemente viene utilizzata nelle interrogazioni, anche se non esiste nella base di dati.
- specificare una tabella contenente un sottoinsieme dei dati di una o più tabella nella base di dati, al fine di restringere l'accesso al solo sottoinsieme contenuto nella vista (sicurezza)

Viste in SQL

Vediamo un esempio di implementazione di una vista in SQL:

Aggiornamento di viste

Le operazioni di aggiornamento sulle viste sono tradotte in opportuni comandi di modifica sulle tabelle di base da cui la vista è derivata. Non sempre è possibile determinare in modo univoco come riportare la modifica sulla vista in termini di modifiche sulle tabelle di base; i problemi sorgono con viste ottenute tramite join di più tabelle.
I sistemi considerano aggiornabili viste definite su una sola tabella, in alcuni casi si richiede che la vista contenga la chiave primaria.

- CHECK OPTION: si applica a viste aggiornabili
- Sono ammessi aggiornamenti sulle righe della vista e dopo l'aggiornamento le righe devono continuare ad appartenere alla vista.
- LOCAL vs CASCADE: profondità con cui applicare i controlli a seguito di aggiornamento

Trigger e basi di dati attive TO DO

Asserzioni

In SQL un ASSERTION consente di specificare un ulteriore vincolo sulla base di dati che non è altrimenti rappresentabile dallo schema.

Sintassi:

```
CREATE ASSERTION <Constraint name>
CHECK (search condition)
[ <constraint attributes> ]
```

Un'asserzione può avere come attributo DEFERRABLE o NOT DEFERRABLE, il primo significa che il DBMS può posticipare il controllo del vincolo fino al termine della transazione, altrimenti controlla il vincolo immediatamente dopo l'esecuzione di ogni SQL statement in una transazione.

Un'asserzione può essere INITIALLY DEFFERED O INITIALLY IMMEDIATE, la prima significa che il vincolo è necessariemente DEFERRABLE e controllato all'inizio di ogni transazione, la seconda significa che il vincolo può essere sia DEFERRABLE o NOT DEFERRABLE e il tempo di controllo è immediato all'inizio di ogni transazione.

vediamo degli esempi:

Trigger

I trigger sono regole attive, essi sono utili per formare basi di dati attive, cioè che hanno un comportamento reattivo, sono in grado di eseguire delle regole oltre che alle transazioni utente.

Lezione 7 - Progettazione fisica

1 Progettazione fisica

1.1 Introduzione

Il concetto di progettazione fisica

Le basi di dati sono memorizzate fisicamente su file di record, i DBMS consentono diverse impostazioni per regolare dei criteri di memorizzazione fisica e ottimizzare il modo in cui i dati vengono salvati. La strategia più semplice per la gestione della memorizzazione fisica consiste nel definire specifici indici sulla base del carico di operazioni previste sui dati.

Collocazione su disco dei record di un file

I dati sono memorizzati sotto forma di **record**. Ogni record è una collezione di valori di dati collegati in cui ogni valore è un insieme di uno o più byte e prende il nome di **campo** del record. I dati di tipo BLOB[1]e CLOB[2], in quanto di grandi dimensioni, non vengono salvati direttamente nel record, quest'ultimo contiene solo un puntatore al dato.

File di record

Un file è semplicemente un insieme ordinato di record. Se tutti i record occupano lo stesso numero di byte, si chiamano record a lunghezza fissa, altrimenti vengono detti record a lunghezza variabile.

Vediamo un esempio in merito, avendo la seguente tabella:

CREATE TABLE country (
iso3 CHAR(3),
currency CHAR(4),
population INTEGER(9),
name VARCHAR(20),)

avremo un record a lunghezza variabile, per via del campo name che è varchar(20), possiamo andare a salvarlo se vogliamo vederlo come record a lunghezza fissa:

1.2 Gestione dei blocchi

Ripartizione in blocchi

Un blocco è un'unità di memoria su disco, di dimensione B byte, un record è un'unità logica di informazione all'interno del file, di dimensione R byte. I nostri record necessitano di essere riportati nei blocchi.

Il fattore di blocco, anche detto blocking factor, è il numero medio di record che possono essere memorizzati all'interno di un singolo blocco, si calcola cosi:

$$\text{bfr} = |B/R|$$

Teniamo presente che se $R \leq B$ potrebbe rimanere spazio non utilizzato, che viene calcolato cosi $B-(bfr*R)$. In questi casi possiamo usare un'organizzazione **spanned**, dove possiamo spezzare i record per utilizzare al meglio gli spazi vuoti che si sono creati, per memorizzare le varie parti di record nei blocchi andiamo ad usare dei puntatori.

Il numero di blocchi necessari per memorizzare un file è $|totalerecord/bfr|$.

[1]Binary Large Object, tipo di dato usato per memorizzare grandi quantità di dati binari
[2]Character Large Object, tipo di dato usato per memorizzare grandi quantità di dati testuali

File di record non ordinati

Organizzazione basata su **file heap** nei quali i record sono memorizzati nell'ordine in cui vengono inseriti, per cui l'inserimento di un recordo è molto efficiente.

lo svantaggio è che questo file richiede una ricerca lineare, in quanto non c'è un ordine.

L'operazione di cancellazione è suddivisa in diversi passi:

- ricerca del record da cancellare
- copiare il blocco corrispondente in un buffer
- cancellare il record
- riscrivere il blocco su disco

In questo modo però avremo uno spreco di spazio sul file, dato dal record cancellato

Un modo più efficace è andare ad associare ad ogni record un bit **indicatore di cancellazione**, cosi i record cancellati vengono marcati tramite questo bit e verranno fisicamente cancellati solo in fase di riorganizzazione del file, in questo modo non avremo spreco di spazio.

File di record ordinati

Un altro metodo di organizzazione dei file, sono i file ordinati, anche detti **file sorted**, nei quali i record sono memorizzati in un ordine specifico, stabilito da un campo detto **campo di ordinamento**.

Cosi facendo i record consecutivi nell'ordinamento sono frequentemente all'interno dello stesso blocco, rendendo molto più efficienti le operazioni di ricerca, ovviamente andando incontro a operazioni di salvataggio dei record più complesse.

Ricerca binaria

La ricerca binaria è utilizzabile in un file ordinato di b blocchi in cui si cerca un record con il valore K nella chiave di ordinamento, l'algoritmo di ricerca è il seguente:

```
sia l = 1; u = b
finchè (u >= l) esegui:
    i = (l+u)/2
    trasferisci il blocco i del file nel buffer
    se K < valore del campo chiave del primo record di i allora u = i-1
    se k > valore del campo chiave dell'ultimo record di i allora l = i+1
    se invece K è nel blocco i terminiamo
```

Inserimento e cancellazione

In un file ordinato, le operazioni di inserimento e cancellazione sono dispendiose.

Una opzione consiste nel tenere in ogni blocco un certo spazio per l'inserimento di nuovi record, oppure possiamo mantenere un file di appoggio non ordinato, detto **file di overflow**, e gli inserimenti verranno effettuati in questo ultimo file che periodicamente verrà unito al file ordinato per aggiornarlo.

File hash

Un file hash si basa sull'idea di organizzare i record in blocchi usando una funzione di hash per determinare in quale blocco memorizzare ciascun record.

Ogni record ha un **campo hash** che viene applicato ad una funzione hash h e lo trasforma nell'indice che corrispone a un blocco, per esempio avendo un numero di slot variabile fra 0 e N-1, la funzione hash trasforma il valore del campo hash in un numero compreso fra 0 e N-1, ad esempio $h(K) = KmodN$.

Lo scopo di una funzione hash è quello di distribuire i record nel modo più uniforme possibile, andando a minimizzare le collisioni fra hash.

Le collisioni possono essere gestite:

- indirizzamento aperto, in caso di posizione occupata si scorre la successiva finchè non si trova una libera
- concatenamento con overflow, in caso di overflow andiamo a salvare il record in una posizione secondaria di riserva
- hash multipli, se avviene collisione si va ad applicare un altra funzione hash

2 Indici

2.1 Caratteristiche generali

Indice

Un indice nei DBMS è una **struttura di accesso secondaria** che permette di trovare i dati più velocemente in una delle strutture fisiche descritte in precedenza.

Gli indici possono essere costruiti con una qualsiasi combinazione di campi dei record e funzionano con lo stesso principio di un indice dei libri: mettono a disposizione una struttura ordinata ad accesso veloce tramite la chiave di indicizzazione e poi utilizzano un puntatore al blocco del record corrispondente per reperire i dati. Il puntatore viene salvato insieme ai valori.

Un indice velocizza le operazioni di accesso, ma causa un maggiore costo di mantenimento in inserimento, cancellazione e modifica, per cui è necessario bilanciare e scegliere con attenzione su quali campi costruire un codice.

Creazione in SQL

La sintassi di creazione di un indice in SQL è la seguente:

```
CREATE INDEX name ON table [USING method] (column)
```

Using method è un campo opzionale che serve a specificare il metodo di accesso, come btree o hash

Tipologie di indici

- **Indice primario**: è creato sul campo chiave di ordinamento di un file ordinato di record, il campo di ordinamento è univoco per tutti i record
- **Indice di cluster**: è creato su un campo non univoco, nei **file clustered** i dati vengono raggruppati in base al valore dell'indice

Un file può avere al massimo un campo di ordinamento fisico, quindi può avere al massimo un indice primario o di cluster, ma non entrambi, ma su un qualsiasi campo non di ordinamento si può andare a definire un **indice secondario**.

2.2 File ordinati

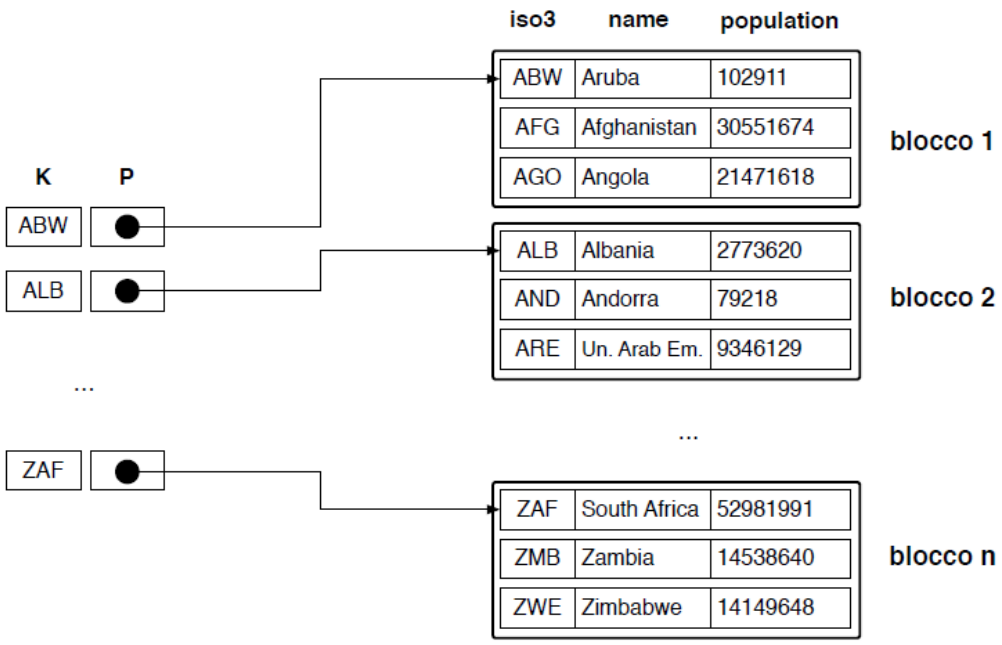
Indici primari

Un **indice primario** è un file ordinato i cui record hanno lunghezza fissa ed è formato solo da due campi:

- un campo chiave, detto anche chiave primaria dell'indice
- un puntatore ad un blocco del disco

Una voce i dell'indice è rappresentato nella seguente forma: $< K(i), P(i) >$.

Vediamo un esempio:



Struttura dell'indice primario

L'indice primario contiene una voce per ogni blocco di dati del file, indipendentemente dal numero di record. Il primo record di ciascun blocco viene detto **record àncora**.

Esistono due tipologie di indici:

- indice denso: contiene una voce per ogni record nel file, veloce per trovare record specifici ma occupa più spazio
- indice sparso: contiene voci per solo alcuni record, ad esempio il primo record di ogni blocco, occupa meno spazio ma più lento nella ricerca

Un indice primario è sempre un indice sparso, in quanto contiene solo la chiave del primo record e il puntatore al blocco ma un blocco contiene più record.

Un file di indice contiene sempre molti meno blocchi rispetto al file dei record, in quanto c'è una voce per ogni blocco del file dei record e i record dell'indice hanno solo due campi, la chiave e il puntatore. Per esempio se un file dei record contiene 1000 record essi saranno salvati in 10 blocchi diversi (presumento 100 record a blocco), di conseguenza il file di indice avrà solamente 10 voci, una per ogni blocco.

Esempio di ricerca binaria sull indice (I)

Supponiamo di avere un file ordinato, senza l'uso di indice, con le seguenti caratteristiche:

- record a dimensione fissa $R = 100$ byte indivisibili
- file con $r = 30000$ record e blocchi $B = 1024$ byte

il fattore di blocco bfr è uguale ad $|B/R| = |1024/100| = 10$ record per blocco

il numero di blocchi necessari per il file è $b = |r/bfr| = |30000/10| = 3000$ blocchi

Possiamo dire che una ricerca binaria sul file di dati richiederebbe aprossimativamente $|\log_2 b| = |\log_2 3000| = 12$ accessi al file

Esempio di ricerca binaria sull indice (II)

Supponiamo di avere 9 byte per record dell'indice, suddivisi in 3 byte per il campo chiave di ordinamento e 6 byte per il puntatore a blocco.

il fattore di blocco per l'indice è $bfr_i = |B/R_i| = |1204/9| = 113$ voci dell'indice per blocco

l'indice è costituito da 3000 voci, in quanto il file dei record occupa 3000 blocchi, perciò il numero di blocchi necessari per il file di indice è $b = |r_i/bfr_i| = |3000/113| = 27$ blocchi

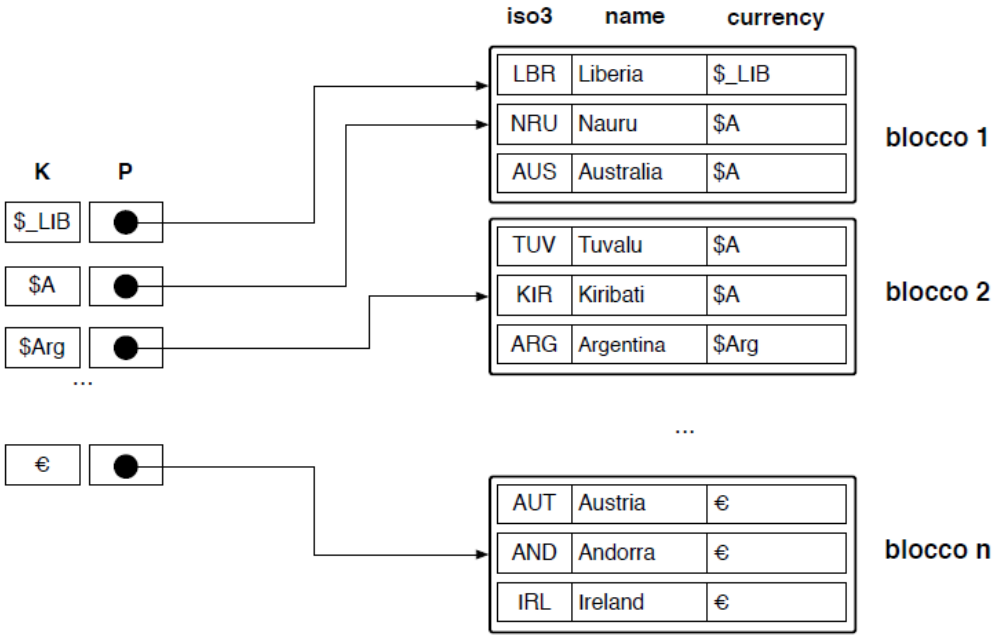
la ricerca binaria richiede quindi $|\log_2 b_i| = |\log_2 27| = 5$ accessi ai blocchi in media

Indice di cluster

Un indice di cluster è usato quando un file è ordinato secondo un campo di ordinamento non chiave e quindi può anche avere valori duplicati

La chiave dell'indice contiene una voce per ogni valore distinto del campo di ordinamento, il puntatore si riferisce al primo blocco che contiene almeno un record corrispondente alla chiave dell'indice, da questo capiamo che anche l'indice di cluster è un indice sparso.

Vediamo un esempio:



2.3 Indici secondari

Indice secondario

Un indice secondario è un'ulteriore struttura di accesso a un file per il quale ci sia già un indice primario.

il file dei record può essere ordinato, ma non sul campo di indicizzazione secondaria, non ordinato od ordinato con hash.

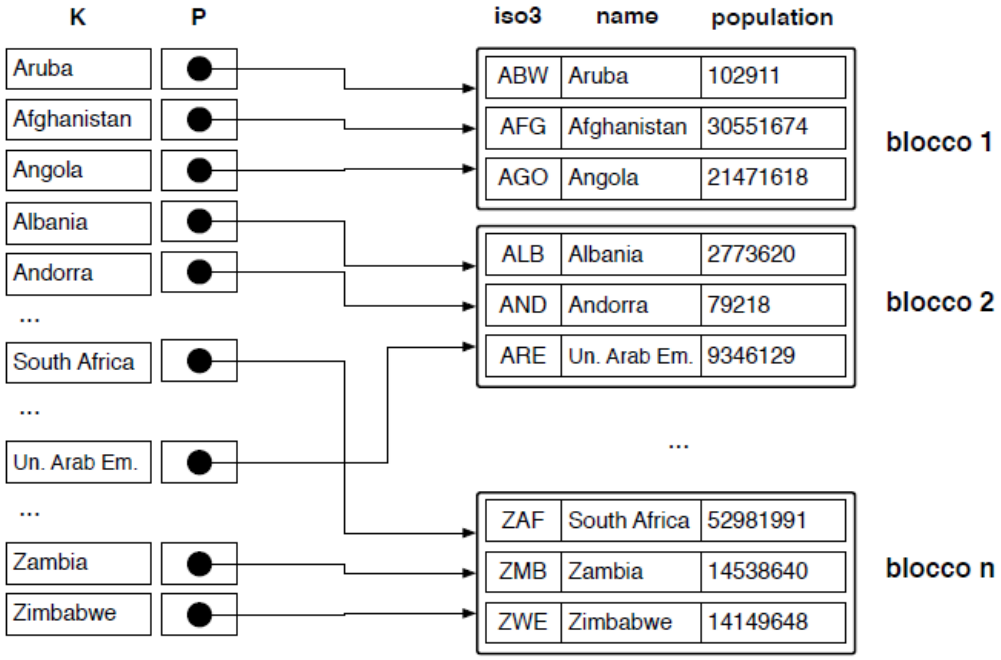
il campo indicizzato può essere una chiave univoca ma non perforza, in termini di SQL la possiamo vedere se il campo indicizzato possiede il vincolo UNIQUE.

Indici secondari su campi chiave

Nel caso il campo di indicizzazione è chiave univoca, l'indice è denso in quanto avremo una voce per ogni record, costituita dalla chiave univoca e un puntatore al record stesso all'interno del blocco.

Nonostante il file contiene molte voci, una per ogni chiave, la ricerca risulta notevolmente più veloce in quanto la chiave dell'indice è mantenuta ordinata e permette una ricerca binaria, mentre senza indice non avremo ordinamento e quindi dovremmo fare affidamento ad una ricerca lineare.

Vediamo un esempio:



Vediamo anche un esempio di numero di accessi:

Consideriamo un file con $r = 30000$ record di dimensione $R = 100$ byte e dimensione blocco $B = 1024$ byte, avremo $b = 3000$ blocchi necessari.

La ricerca lineare su un campo non ordinato richiede mediamente $b/2 = 3000/2 = 1500$ accessi

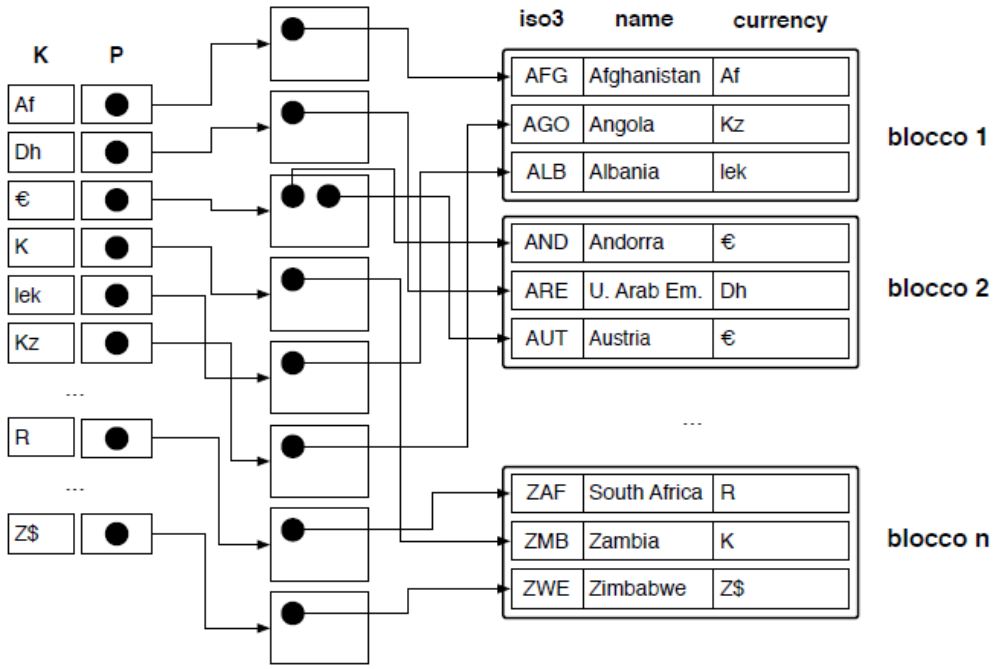
Tramite un indice secondario, supponendo che il suo record abbia dimensione $R_i = 15$ byte avremmo 30000 voci nell'indice con una occupazione di 442 blocchi, dato che $bfr_i = |1024/15| = 68$. In questo caso possiamo affidarci alla ricerca binaria che ha una media di $|\log_2 442| = 9$ accessi, più un ulteriore accesso al blocco del puntatore, quindi un totale di 10 accessi rispetto ai 1500 precedenti.

Indici secondari su campi non chiave

In questo caso il campo indicizzato non è univoco, può avere duplicati, abbiamo tre opzioni di memorizzazione dell'indice:

- inserire più voci dell'indice con medesimo valore di K
- usare un record a lunghezza variabile per l'indice in modo da inserire più puntatori per ogni voce dell'indice K univoca (indice sparso)
- mantenere voci a lunghezza fissa, ma inserendo un ulteriore livello per i puntatori

Vediamo un esempio della terza opzione:



Classificazione per campo di indicizzazione

	Campo indice ordinato	**Campo indice non ordinato**
Chiave	primario	secondario(chiave)
Non chiave	cluster	secondario(non chiave)

Classificazione per proprietà

Indice	**Numero Voci**	**Denso**	**Oggetto del puntatore**
primario	numero blocchi	no	blocco
cluster	numero valori univoci	no	blocco
secondario chiave	numero record	si	record/blocco
secondario non chiave	numero record	si/no	blocco

3 Indici multilivello

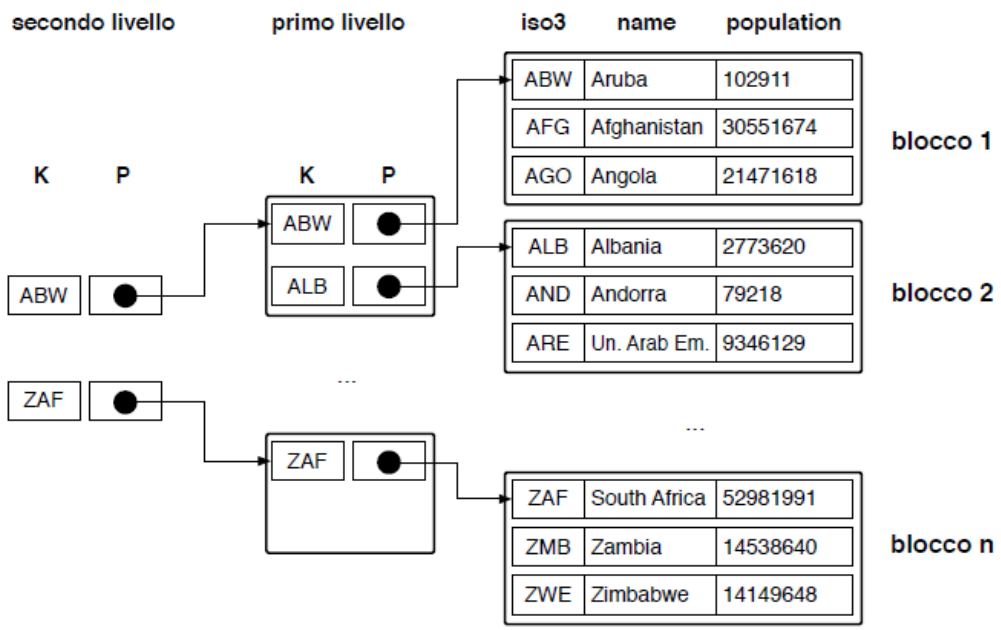
3.1 Introduzione

L'idea alla base della ricerca binaria è di dividere per due ad ogni passo lo spazio di ricerca del valore desiderato, da questo deriva la stima di costo $\log_2 b_i$.

Usando un indice multilivello, andiamo a crerare un indice primario del file di indice. Definiamo **fan-out (fo)** dell'indice di primo livello il suo blocking factor bfr_i, cosi otteniamo un indice di secondo livello che ha un costo di accesso $\log_f ob_i$, il motivo di ciò è che a ogni passo della ricerca, usando l'indice di secondo livello, siamo in grado di divere lo spazio di ricerca di un fattore pari al numero di voci del blocco di file di indice.

In questo modo possiamo usare più livelli di indicizzazione a parte da indici di qualsiasi tipo purchè il campo chiave K dell'indice di primo livello contenga valori univoci.

Vediamo un esempio:



Vediamo anche un esempio di numero di accessi:

Consideriamo l'esempio dell'indice secondario con $bfr_i = 68$ e $b_1 = 442$ blocchi.

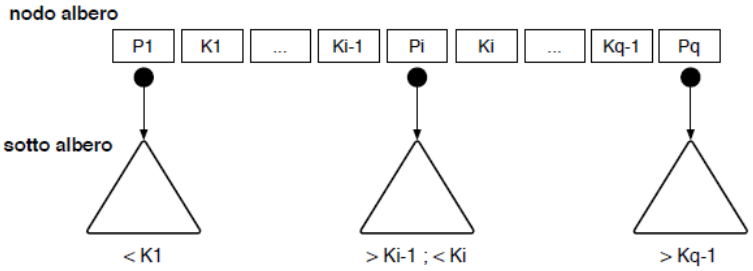
Il numero di blocchi al secondo livello sarà $b_2 = |b_1/bfr_i| = |442/68| = 7$ e il numero di blocchi al terzo livello sarà $b_3 = 1$.

In questo modo, per accedere a un record sarà necessario accedere a un blocco per ogni livello più il blocco del file di dati, per un totale di $3 + 1 = 4$ accessi. $\log_2 7 = 3$ arrotondando per eccesso.

Lo schema di indicizzazione usando un file ordinato e un indice primario multilivello è la più usata e prende il nome di **file sequenzialmente indicizzato**.

Alberi di ricerca

Un **albero di ricerca** di ordine p è un albero tale per cui ogni nodo contiene al massimo $p - 1$ valori di ricerca e i p puntatori ai sottoalberi sono nell'ordine: $< p_1, K_1, p_2, K_2, \cdots, P_{q-1}, K_{q-l,P_p} >$

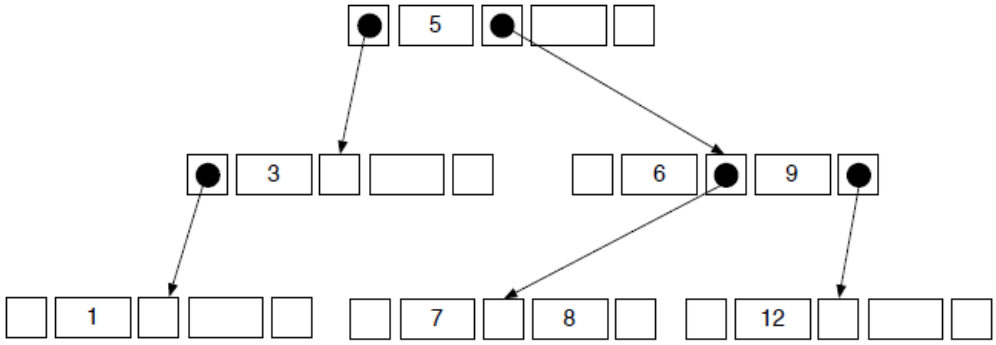


I vincoli da rispettare sono:

- in ciascun nodo $K_1 < K_2 < \cdots < K_{q-1}$, chiavi ordinate in modo crescente
- per tutti i valori X del sottoalbero a cui si riferisce P_i si ha $K_i - 1 < X < K_i$

Vediamo un esempio:

Albero di ricerca di ordine p=3



Bilanciamento degli alberi di ricerca

La caratteristica più importante nella gestione di un albero di ricerca è mantenere il suo bilanciamento in modo che:

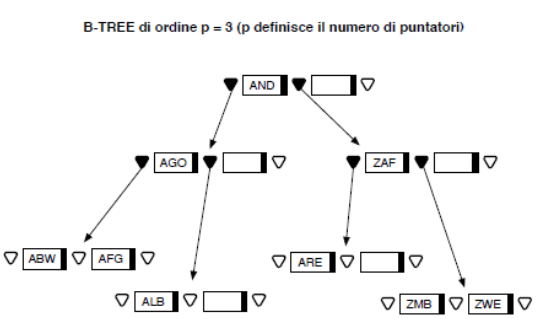
- i nodi siano distribuiti uniformemente e la profondità dell'albero sia minimizzata
- rendere uniforme la velocità di ricerca in modo che il tempo medio per trovare una qualsiasi chiave sia lo stesso

3.2 Alberi B-tree

Un albero B-tree (balanced tree) è un albero di ricerca con ulteriori vincoli che ne garantiscono il bilanciamento. Un B-tree di ordine p usato come struttura di accesso su un campo chiave di un file di dati ha la seguente struttura:

- ogni nodo interno ha forma: $< P_1, < K_1, Pr_1 >, P_2, < K_2, Pr_2 >, ..., < K_{q-1}, Pr_{q-1} >, P_q >$ in cui Pr_i è il puntatore al record o blocco corrispondente a K_i, mentre p_i è un puntatore al sotto albero, K_i son le chiavi ordinate
- all'interno di ogni nodo $K_1 < K_2 < \cdots < K_{q-1}$
- per tutti i valori X del campo chiave nel sottoalbero puntato da P_i si ha: per $1 < i < q$ vale $K_i - 1 < X < K_i$; per $i = 1$ vale $X < K_i$; per $i = q$ vale K_{i-1}
- ogni nodo contiene al massimo p puntatori dell'albero
- ogni nodo tranne radice e foglie ha almeno $p/2$ puntatori dell'albero; il nodo radice ha almeno due puntatori all'albero salvo che sia l'unico nodo dell'albero
- un nodo con q puntatori a albero (con $q \leq p$) contiene $q - 1$ valori del campo chiave e quindi $q - 1$ puntatori ai dati
- tutti i nodi foglia sono allo stesso livello e hanno la medesima struttura dei nodi intermenti a parte il fatto che tutti i loro puntatori a albero sono nulli

Vediamo un esempio:



FILE DATI

ABW	Aruba	102911
AFG	Afghanistan	30551674
AGO	Angola	21471618
ALB	Albania	2773620
AND	Andorra	79218
ARE	Un. Arab Em.	9346129
ZAF	South Africa	52981991
ZMB	Zambia	14538640
ZWE	Zimbabwe	14149648

LEGENDA

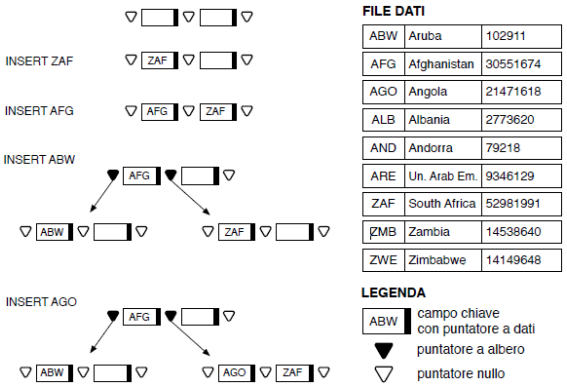
ABW	campo chiave
●	puntatore a dati
▲	puntatore a albero
△	puntatore nullo

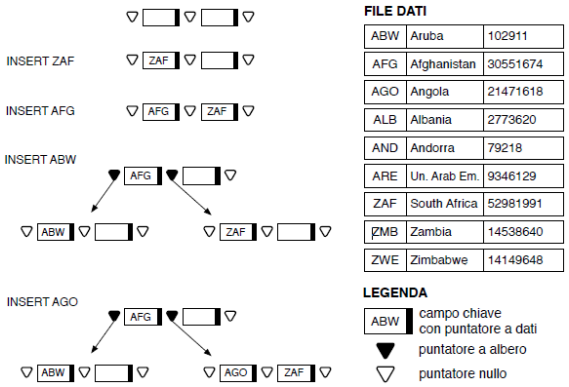
inserimento in B-tree

Andiamo a trovare il nodo foglia che dovrebbe contenere il nuovo valore di inserimento, se il nodo contiene meno di $p-1$ chiavi, inserire il nuovo elemento e ordinare le chiavi e i rispettivi puntatori, se il nodo non ha più spazio, dividerlo equamente in due nodi come segue:

- trovare il valore intermedio nella lista formata dalle chiavi del nodo più il nuovo valore
- inserire i valori strettamente inferiori al valore intermedio in un nodo di sinistra e i valori strettamente superiori al valore intermedio in un nodo di destra
- inserire il valore intermedio nel nodo di ordine superiore, causando se necessario l'aggiornamento ricorsivo dei nodi superiori per mezzo della stessa operazione di divisione, se il nodo di ordine superiore è la radice dell'albero e non può contenere nuovi valori, allora procedere alla creazione di una nuova radice

Vediamo due esempi in merito:





Ricerca

La ricerca su B-tree è simile alla ricerca su albero binario; si cerca il valore desiderato nel nodo radice dell'albero, se il valore non è stato trovato, si ripete ricorsivamente l'operazione sull'albero a cui si riferisce il puntatore a sinistra del primo valore maggiore alla chiave, se non ci sono valori maggiori della chiave di ricerca si procede con il puntatore a destra dell'ultimo valore del nodo.

3.3 Alberi B^+

In un albero B^+ i puntatori ai dati sono memorizzati solo nei nodi foglia, perciò i nodi foglia contengono tutti i valori, mentre i nodi intermedi contengono solo alcuni valori K che sono usati per guidare la ricerca come in un B-tree. Inoltre i nodi foglia contengono degli ulteriori puntatori che permettono di accedere da una foglia direttamente alla successiva.

Inserimento

L'inserimento avviene come nei B-tree, con una sola differenza, se il nodo richiede di essere diviso, in caso fosse un nodo intermedio si procede come nei B-tree, mentre se è una foglia si divide nel seguente modo:

- creare due nodi foglia
- inserire i primi $(n-1)/2$ valori nel primo nodo
- inserire i rimanenti nel secondo
- inserire nel nodo padre un nuovo puntatore per il secondo nodo foglia e sistemare i valori chiave del nodo padre
- se il nodo padre è pieno, propagare l'operazione di divisione verso l'alto

Ricerca

La ricerca per una chiave K si esegue tramite questi passaggi:

- cercare nel nodo radice il più piccolo valore di chiave maggiore di K

- se il valore esiste, seguire il puntatore subito precedente, altrimenti seguire l'ultimo puntatore del nodo
- se raggiungiamo un nodo foglia, cercare K nel nodo, altrimenti ripetere il passo successivo

Vantaggi

il fatto di non avere puntatori diretti ai dati nei nodi intermedi può apparire svantaggioso rispetto all'uso dei B-tree, poichè nei nodi intermedi occorre memorizzare solo puntatori ai sottoalberi e chiavi e non anche i puntatori ai dati, il numero di puntatori a sottoalberi memorizzabili in un nodo intermedio di un albero B^+ è superiore rispetto a un B-tree, ciò comporta un vantaggio in termini di accessi al disco necessari per attraversare l'albero.

Il fatto che i nodi foglia siano linkati in modo da formare di fatto un indice ordinato permette di combinare i vantaggi di un indice multilivello con i vantaggi del metodo di accesso sequenziale indicizzato (ISAM).

Lezione 8 - Elementi di sicurezza

1 Politiche e modelli

1.1 Introduzione

Obiettivi

- Segretezza: impedire a persone non autorizzate la lettura delle informazioni
- Integrità: impedire a persone non autorizzate di eseguire modifiche e/o cancellazioni sui dati
- Disponibilità: garantire l'accesso alla persone autorizzate ai dati relativi

Teniamo presente che l'importanza di questi obiettivi varia a seconda del database di riferimento.

Tecniche

- Autenticazione: meccanismi per verificare l'identità dell'utente che si connette al sistema
- Controllo dell'accesso: meccanismi che verificano che l'utente sia autorizzato a compiere l'accesso ai dati richiesti
- Crittografia: meccanismi che cifrano i dati in modo che possano essere decifrati solo da utenti autorizzati

Controllo dell'accesso

Il controllo dell'accesso regola le operazioni che utenti e programmi possono eseguire su informazioni e risorse del database. Lo scopo è limitare e controllare le operazioni che gli utenti effettuano, in modo da prevenire errori o danni all'integrità e alla segretezza dei dati.

Le risorse sono costituite dai dati e sono memorizzate in oggetti a cui si vuole garantire protezione.

I soggetti sono agenti, cioè utenti o programmi, che richiedono di poter esercitare privilegi sui dati, come operazioni di lettura, scrittura o esecuzione.

Le politiche di sicurezza sono regole generali stabilite dall'organizzazione realtive alla sicurezza dei dati nel database. Queste regole vengono implementate tramite regole di autorizzazione che vanno a stabilire i privilegi per ogni utente può esercitare sui vari oggetti del sistema.

Il **reference monitor** è un meccanismo di controllo che ha il compito di stabilire se l'utente può essere autorizzato a compiere l'accesso.

1.2 Politiche di sicurezza

La politica di sicurezza adottata dipende da fattori organizzativi, come l'ambiente di installazione, le esigenze degli utenti, i regolamenti dell'organizzazione e vincoli di natura legale.

Esisto due classi fondamentali:

- Politiche per l'amministrazione della sicurezza
- Politiche per il controllo dell'accesso ai dati

Politiche per l'amministrazione

Queste politiche sono regole che stabiliscono che concede e revoca i diritti di accesso, ci sono due modelli principali:

- Amministrazione centralizzata: un unico autorizzatore, detto DBA, controlla l'intera base di dati
- Amministrazione decentralizzata: più autorizzatori sono responsabili del controllo di varie porzioni della base di dati

Ownership è l'utente che crea un oggetto, quindi il proprietario, e gestisce le autorizzazioni su di esso.

Controllo degli accessi

Le politiche per il controllo dell'accesso stabiliscono quali soggetti possono accedere a determinati dati contenuti nel sistema, quali operazioni possono fare se questi dati e come vengono eventualmente trasmessi i diritti di accesso.

Esistono due principali politiche in merito:

- Need To Know (minimo privilegio): ogni utente può accedere solo alle informazioni necessarie per svolgere il proprio lavoro, quindi un accesso molto restrittivo
- Maximized Sharing (massima condivisione): permette agli utenti il massimo accesso possibile alle informazioni, pur mantenendo la riservatezza delle informazioni

Caratteristiche

- Need To Know: offre ottime garanzie di sicurezza, è adatta a database con forti esigenze di protezione e può portare ad un sistema eccessivamente protetto, andando anche a negare accessi che non comprometterebbero la sicurezza del sistema.
- Maximized Sharing: viene utilizzata in ambienti in cui esiste una certa fiducia tra gli utenti ed in cui non è sentita una forte esigenza di protezione.

Sistemi

Nei database esistono due tipologie di sistemi:

- Sistema aperto: l'accesso è permesso a tutti, a meno che non ci sia una regola che lo nega, infatti per ogni soggetto si elencano i diritti vietati, dette eccezioni, cioè gli viene detto quello che può fare tutto tranne quello esplicitamente negato.
- Sistema chiuso: l'accesso è permesso solo se esplicitamente autorizzato dalle regole di autorizzazione, le quali indicano per ogni soggetto i diritti concessi, cioè gli viene detto che non può fare nulla eccetto determinate operazioni permesse.

Un sistema chiuso si basa sulla politica del minimo privilegio, mentre un sistema aperto si basa sulla politica della massima condivisione. Un sistema chiuso ha il vantaggio di offrire maggiori garanzie di sicurezza, infatti una regola cancellata erroneamente o non inserita restringe ulteriormente l'accesso, mentre un sistema aperto permette accessi non autorizzati. Ai giorni nostri la maggior parte dei database esistenti sono sistemi chiusi.

Granularità

La granularità indica a che livello vengono applicati i controlli di accesso.

Il requisito minimo, ricade nalla possibilità di specificare nelle regole di autorizzazione sugli oggetti a cui l'utente può accedere, nei database relazionali si fa riferimento alle relazioni o attributi delle relazioni.

Tipologie di controllo

Esistono varie tipologie di controllo:

- controllo dipendente dal nome: l'accesso si basa sul nome dell'oggetto (tabella, vista, colonna) ed è il più comune e standard
- controllo dipendente dal contenuto: l'accesso si basa sul valore di uno o più attributi, si usa spesso nelle viste; per esempio "mario può vedere solo i dati degli impiegati con stipendio minore di una determinata soglia"
- controllo dipendente dal contesto: l'accesso si basa sulle variabili di sistema, come data, ora e luogo; un esempio può essere che gli impiegati possono accedere ai dati solamente durante l'orario lavorativo
- controllo dipendente dalla storia degli accessi: l'accesso dipende da cosa l'utente ha gia fatto in passato, per esempio un determinato utente può accedere al file solo se non ha già effettuato più di tre accessi nell'ultima ora

Politiche discrezionali

Ogni soggetto, che sia utente o processo, ha un set di diritti su determinati oggetti, questi sono gestiti tramite regole di autorizzazione. Gli utenti possono, a loro discrezione, concedere o revocare i diritti di accesso sugli oggetti.

Il vantaggio è dovuto dalla alta flessibilità, che rende il tutto adattabile a numerosi scenari

lo svantaggio è che non vengono imposte restrizioni sull'uso che viene fatto del dato una volta acceduto, ovvero non c'e alcun controllo sul flusso di informazioni nel sistema. Un flusso di informazioni nel sistema tra un oggetto X e un oggetto Y si verifica quando si effettua una lettura del valore di X e una scrittura del valore in Y.

Politiche mandatorie

In caso di basi di dati governative le politiche discrezionali non sono adatte, in quanto sono presenti informazioni vitali, diversi livelli di sensitività delle informazioni e i controlli sul flusso di dati sono essenziali. Se si utilizzasse politiche discrezionali si possono verificare attacchi sofisticati da parte di utenti determinati, un esempio di attacco può essere il Cavallo di Troia.

Le politiche mandatorie regolano l'accesso tramite la definizione di classi di sicurezza per i soggetti e gli oggetti del sistema.

Le classi di sicurezza sono ordinati parzialmente da una relazione d'ordine.

La classe di sicurezza assegnata ad un oggetto indica il livello di sensività dell'oggetto: maggiore è la classe assegnata, maggiore è il danno causato se si verifica la fuga dei dati dell'oggetto.

La classe di sicurezza che viene assegnata ad un utente, è una misura del grado di fiduca che stiamo dando a tale utente nel non commettere violazioni.

Controllo dell'accesso

Il controllo dell'accesso è regolato da una serie di assiomi di sicurezza che stabiliscono le relazioni che devono essere verificate fra la classe di un soggetto e quella di un oggetto affinchè al primo sia concesso di esercitare un modo di accesso sul secondo.
Queste politiche vengono applicate in ambienti dove la quantità di informazioni da proteggere è elevata, ci sono forti esigenze di protezione ed è possibile classificare rigidamente gli elementi del sistema.
I sistemi che adottano una politica mandatoria sono spesso indicati come sistemi multilivello.

Controllo del flusso

Le politiche mandatorie possono essere usate anche come politiche per il controllo del flusso, poichè evitano che le informazioni una volta accedute vengano trasferite verso oggetti con classificazione inferiore e quindi più accessibili. In questo modo c'è un controllo completo sul sistema di autorizzazione ma la flessibilità risulta ridotta e la circolazione di informazioni tra gli utenti è più difficile.

Le politiche mandatorie e discrezionali non sono mutuamente esclusive, possono essere utilizzate insieme. In questo caso la politica mandatoria controlla come vengono assegnate le autorizzazioni mentre la politica discrezionale spetta il compito di controllare le richieste di accesso.

Cavallo di troia

Un esempio di Cavallo di Troia può essere il seguente: un utente Y è proprietario di un file F2, esso concede all'utente X di accedere in scrittura su F2, l'utente X è proprietario del file F1, quest'ultimo lancia un programma P che contiene del codice nascosto che va a copiare le informazioni da F1 nel file F2, in questo modo Y può accedere al contenuto di F1 senza avere i permessi.

2 Modelli per il controllo dell'accesso

2.1 System R

Questo modello implementa una politica discrezionale e supporta il controllo dell'accesso in base al nome (relazione o attributo) e al contenuto (determinate righe).

Questo sistema è un sistema chiuso nel quale un accesso è concesso solo se esiste una esplicita regola che lo autorizza.

L'amministrazione dei privilegi è decentralizzata mediante ownership, cioè quando un utente crea una relazione, riceve automaticamente tutti i diritti di accesso su di essa ed anche la possibilità di delegare ad altri tali privilegi.

GRANT

La sintassi base del comando GRANT è la seguente:

```
GRANT lista privilegi | ALL [privilegi] ON lista relazioni/viste
TO lista utenti|PUBLIC [WITH GRANT OPTION];
```

Opzioni del GRANT

Il GRANT ha come oggetti di protezione le relazioni e le viste, non lavora sugli indici o trigger.

I privilegi che possiamo usare sono: INSERT, SELECT e UPDATE, in questo caso solo il proprietario può cancellare un oggetto.

La parola chiave ALL (ALL PRIVILEGES) consente di concedere con un solo comando tutti i privilegi su una determinata relazione, da tenere presente che non è utilizzabile sulle viste. Tramite un unico comando di GRANT si possono concedere più privilegi su una stessa relazione e concedere privilegi sulla stessa relazione a più utenti.

Il comando GRANT con soggetto PUBLIC permette una concessione di privilegi a tutti gli utenti.

GRANT OPTION

La GRANT OPTION è un campo opzionale del comando GRANT che permette a chi riceve un privilegio di esercitare il privilegio ricevuto e di concederlo a sua volta ad altri utenti. Un utente può concere un privilegio di una determinata relazione solo se è il proprietario della relazione o se ha ricevuto tale privilegio tramite GRANT OPTION. In caso la clausola WITH GRANT OPTION non fosse specificata l'utente che riceve i privilegi non può concederli ad altri utenti.

I privilegi che ogni utente possiede sono divisi in:

- delegabili: privilegi concessi con grant option
- non delegabili: privilegi concessi senza grant option

Implementazione

sysauth e **syscolauth**: le regole di autorizzazione specificate dagli utenti sono memorizzate in due cataloghi di sistema detti sysauth e syscolauth, implementati come relazioni.

Sysauth gestitsce i privilegi a livello di tabella o vista, una tupla di sysauth ha i seguenti attributi:

- id_utente: id dell'utente a cui sono concessi i privilegi
- nome: nome della relazione su cui sono concessi i privilegi
- creatore: utente che ha creato la relazione
- tipo $\in R, V$: indica se l'oggetto è una relazione (tipo = 'R') o una vista (tipo = 'V')
- P $\in Y, N$: indica se l'oggetto ha (Y) o meno (N) il privilegio sulla relazione

- update $\in ALL, SOME, N$: indica se il soggetto ha il privilegio di update su tutte (ALL), alcune (SOME) o nessuna (N) colonna della relazione
- grantopt $\in Y, N$: indica se i privilegi sono delegabili (Y) o meno (N)

Syscolauth gestisce i privilegi di UPDATE a livello di colonna, una tupla di syscolauth ha i seguenti attributi (id_utente, nome, colonna, grantopt) per ogni colonna della relazione su cui l'utente identificato da id_utente può esercitare il privilegio di update.

Uso del catalogo

Quando un utente x esegue un comando di GRANT, il meccanismo di controllo accede ai cataloghi di sysauth e syscolauth per determinare se x ha il diritto di delegare i privilegi specificati nel comando.

L'insieme dei privilegi delegabili che l'utente x possiede è intersecato con l'insieme dei privilegi che vuole assegnare nel comando di GRANT. Se tale intersezione risulta vuota, il comando GRANT viene rifiutato, mentre se l'intersezione coincide con i privilegi specificati nel comando, vengono concessi tutti i privilegi specificati, in caso di una intersezione parziale il comando viene eseguito parzialemente solo con i privilegi contenuti nell'intersezione.

REVOKE

```
REVOKE Lista Privilegi | ALL [PRIVILEGES] ON Lista Relazioni | Lista Viste
FROM Lista Utenti | PUBLIC
```

L'unico utente che può revocare i privilegi che ha concesso è ste sesso, è possibile andare a recovare più privilegi con un comando di REVOKE ed un unico comando di REVOKE può essere utilizzato per recovare gli stessi privilegi sulla stessa relazione ad utenti diversi in contemporanea.

Bisogna tenere presente che l'utente che subisce l'operazione di revoca perde tali privilegi, tranne se questi privilegi provengono anche da altre sorgenti indipendenti da quella che effettua la revoca.
Per esempio se Bianchi fornisce permessi con grant option a rossi e verdi, in un secondo momento sia rossi che verdi danno lo stesso privilegio a gialli, ma ad un certo punto verdi decide di revocare il suo privilegio, gialli continuerà ad avere il privilegio per via della concessione di rossi.

Revoca ricorsiva

L'operazione di revoca è ricorsiva, quando un utente revoca un privilegio, non si limita a togliere quel privilegio solamente all'utente che l'aveva concesso, ma fa anche una pulizia di tutti i privilegi a catena che quell'utente aveva ridistribuito grazie al privilegio ricevuto. Possiamo dire che l'operazione di revoca ha l'effetto di modificare il sistema portandolo in uno stato equivalente a quello di partenza, cioè prima che venisse concesso il potere all'utente interessato dalla revoca.

Problema della revoca

Un problema dell'operazione di revoca è che il DBMS deve capire se l'utente che perde il privilegio lo aveva anche da un altra fonte indipendente, per risolvere ciò Sysauth e Syscolauth sono modificati per mantenere, per ogni privilegio, anche l'utente che ha concesso il privilegio, denominato grantor.

Ogni colonna relativa ad un tipo di privilegio in Sysauth contiene (invece di Y e N) un timestamp che denota il tempo in cui il privilegio è stato concesso. Il valore 0 indica che l'utente non ha quel privilegio mentre un valore t $\neq 0$ indica che il privilegio è stato garantito all'utente al tempo t. I privilegi garantiti con lo stesso comando di GRANT hanno lo stesso timestamp.

2.2 Autorizzazioni su viste

Le vise sono molto utili per gestire il controllo dell'accesso ai dati nei database, consentono di:

- delegare privilegi solo su certe colonne; cioè creare una vista come proiezione sulle colonne di cui si voglio concedere i privilegi
- delegare privilegi basati su condizioni specifiche; per esempio autorizzare un utente a visualizzare solo i dipendenti che guadagnano più di 2000 euro
- delegare privilegi statistici; è possibile creare viste che calcolano operazioni come media, somma, conteggio

Quando un utente crea una vista, i privilegi che può esercitare su di essa dipendono da:

- la semantica della vista, cioè la sua definizione in termini di colonne, tabelle o altre viste che usa
- i privilegi che il creatore possiede sulle relazione o viste usate
- un privilegio su una vista può essere delegato solo se il creatore della vista ha il diritto di delegare quei privilegi su tutte le relazioni usate

Tipologie

Vista V definita su una singola relazione R: il proprietario di V ha su V gli stessi privilegi che ha su R, tranne quei privilegi che non si possono esercitare sulla vista a causa della sua semantica.
Per esempio UPDATE o INSERT potrebbero non essere possibili se la vista include condizioni WHERE o omette chiavi primare.

Vista V definita su più relazioni: il proprietario di V ha su V l'intersezione dei privilegi che possiede su ciascuna delle relazioni coinvolte, tranne quei privilegi che non si possono esercitare sulla vista a causa della semantica.

Lezione 9 - Cenni su transazioni

1 Transazione

1.1 Introduzione

Una transazione è un insieme di operazioni sui dati che vengono considerate come un'unica azione. Quindi o vengono tutte completate con successo oppure se anche solo una delle operazioni interne fallisce, nessuna delle operazioni ha effetto sul database. Un aspetto fondamentale è che le operazioni interne alla transazione e il loro esito non sono visibili non sono visibili a altre transazioni, ma saranno visibili solo a transazione conclusa.

Utilità delle transazioni

Mantengono il corretto funzionamento del database anche in presenza di singole operazioni che falliscono ed anche in presenza di operazioni concorrenti, ovvero operazioni eseguite in contemporanea sugli stessi dati.

Una transazione parte da uno stato corretto di una database e si conclude in un altro stato corretto del database. Durante l'esecuzione di quest'ultima, lo stato può essere non corretto temporaneamente. Al termine della transazione vi sono solo due siti possibili:

- COMMIT: nel caso fosse stato raggiunto uno stato corretto, che viene salvato
- ROLLBACK: nel caso ci fosse stato un errore, allora si torna ad uno stato corretto precedente

Uso delle transazioni

Vogliamo usare le transazioni nel caso che in una sequenza di operazioni che vengono eseguite correttamente eccetto una che fallisce e va a causare inconsistenza sui dati, in questo caso nessuna variazione deve avvenire nel database.

Per esempio durante l'operazione di prelievo di 100 euro da un conto, vi sono tre operazioni:

- verifica della disponibilità dei soldi sul conto
- rimozione della cifra indicata
- fuoriuscita dei soldi dallo sportello

Supponiamo il caso che lo sportello abbia un malfunzionamento, la transazione ci permette di non vedere l'addebito dei soldi prelevati e ritornare allo stato di partenza.

Effetto della transazione

L'uso di una transazione fa si che una volta raggiunta l'istruzione di COMMIT, il DBMS verifichi se si è generato un errore dall'istruzione di inserimento, nel nostro caso, e in caso di errore effettui il ROLLBACK, riportando lo stato dell'intero database a prima dell'inizio della transazione.

1.2 Problemi di concorrenza

Problemi dovuti ad accesso concorrente

Quando più utenti o applicazioni lavorano contemporaneamente sugli stessi dati, possono nascere problemi in quanto vengono effettuate operazioni di lettura e scrittura sugli stessi file in memoria.

Le operazioni fondamentali di un DBMS sono:

- read: leggere i dati dalla memoria secondaria nella memoria principale
- write: scrivere i dati della memoria principale nella memoria secondaria

Le principali anomalie che si possono verificare durante operazioni concorrenti sono:

- lost update
- dirty read

- incorrect summary

Lost update

Succede quando due programmi che accedono agli stessi dati, eseguono le proprie operazioni in tempi tali che il valore finale di qualche dato risulta errato.

Dirty read

Succede quando un programma P1 aggiorna un certo dato X e successivamente fallisce e il dato X aggiornato in modo errato viene letto da un altro programma P2 prima che X venga riportato al suo dato iniziale, in questo caso il programma P2 lavora su un dato errato.

Incorrect summary

Succede quando un programma calcola una funzione aggregata su un insieme di record, mentre un altro programma effettua un aggiornamento sui record, il primo programam può calcolare la funzione aggregata considerando alcuni valori precedenti all'aggiornamento e alcuni valori successivi, causando dei dati incoerenti come risultato della funzione.

Proprietà delle transazioni

Il DBMS esegue le transazioni concorrenti in modo tale da garantire che queste ultime soddisfino le seguenti proprietà, dette proprietà ACID:

- Atomicity
- Consistency
- Isolation
- Durability

Comportamento di default

Nella modalità AUTOCOMMIT ogni operazione SQL viene considerata come una transazione, non serve specificare esplicitamente BEGIN e COMMIT. Mentre in caso di uso esplicito di COMMIT o ROLLBACK la transazione corrente termina e ne inizia una nuova.

Atomicity

Implica che una transazione venga eseguita nella sua interezza oppure non venga eseguita, cioè o tutte le operazioni fanno a buon fine e si effettua il COMMIT, oppure se qualche operazione dovesse fallire si effettua il ROLLBACK per cancellare le modifiche parziali eseguite.

Il ROLLBACK può avvenire in modo automatico in caso di errore oppure può essere esplicitato dall'utente per catturare eventuali condizioni logiche non soddisfatte.

Consistency

Ogni transazione parte sempre da un database che rispetta tutti i vincoli di integrità, alla sua conclusione questi vincoli di integrità devono essere tutti soddisfatti.

I vincoli possono essere controllati in due modi:

- IMMEDIATE: la verifica dei vincoli viene eseguita durante l'esecuzione della transazione, l'operazione che viola un vincolo viene cancellata (UNDO) senza andare ad imporre un abort dell'intera transazione
- DEFERRED: la verifica dei vincoli viene eseguita al termine della transazione, dopo il COMMIT. In caso di violazione di qualche vincolo viene eseguito il ROLLBACK della transazione, tornando allo stato precedente.

Isolation

In caso di più transazioni che lavorano in contemporanea, il database si comporta come se ogni transazione venisse eseguita in modo indipendente durante tutta la sua vita. Il DBMS può andare ad alternare l'esecuzione dei comandi delle varie transazioni per velocizzare l'esecuzione ma il risultato deve rimanere lo stesso come se si fosse eseguito transazione per transazione singolarmente.

Durability

Una volta che una transazione è stata completata con successo, quindi è stato effettuato il COMMIT, le modifiche ai dati sono permanenti anche in caso di guasti al sistema.